

STAT 8678 - SAS Programming & Data Analysis

Chi-Kuang Yeh

2026-03-03

Table of contents

Preface	7
Description	7
Prerequisites	7
Instructor	7
Office Hour	7
Grade Distribution	8
Assignment	8
Midterm	8
Topics and Corresponding Lectures	8
Recommended Textbooks	9
Acknowledgments	9
I Introduction	10
1 Introduction to Basic SAS Operation	11
1.1 Introduction to SAS	11
1.1.1 SAS Installzation	12
1.1.2 SAS Windows	13
1.2 SAS Program	15
1.3 SAS Dataset	17
1.4 SAS Examples	17
1.4.1 Creating a Dataset	17
1.4.2 Sorting Data	18
1.5 Generating Summary Statistics	19
1.6 Other notes	19
1.7 Data Type	20
2 Working with SAS Syntax	23
2.1 SAS Statments	24
2.2 Diagnosing and Correcting Syntax Errors	27
2.3 Solution to the example	28
3 Import and Export Dataset	30
3.1 Reading from External Files	30

3.2	Export a File from SAS	32
3.3	Import & Export in SAS Virtual Studio	32
3.4	Solution to the example	34
4	Random Variables	35
4.1	What Is a Random Variable?	35
4.2	Discrete vs Continuous Random Variables	36
4.2.1	Discrete Random Variables	36
4.3	Common Discrete Distributions	37
4.3.1	Bernoulli Distribution	37
4.3.2	Poisson Distribution	37
4.4	Continuous Random Variables	37
4.4.1	Normal Distribution	38
4.4.2	Exponential Distribution	39
4.5	Solution to the exercise	40
II	Statistical Analysis	41
5	Introduction to Statistical Inference I	42
5.1	Probability versus Statistics	42
5.2	Example: One-Sample Mean Problem	42
5.2.1	Problem Formulation	43
5.2.2	Formal Definitions	44
5.2.3	Confidence Interval for μ	45
5.2.4	Hypothesis Testing for μ	45
5.3	Revisit the data example by using SAS	46
5.4	More about hypothesis and confidence intervals	50
5.4.1	How to construct a confidence interval?	50
5.4.2	Rearranging the Inequality	51
5.4.3	Large-Sample Confidence Interval	51
5.5	3.2 Hypothesis Testing and Confidence Intervals Always Agree	51
5.5.1	Interpretation of the p -value	51
5.5.2	Computing the p -value	52
5.5.3	Connection to Confidence Intervals	52
6	Introduction to Statistical Inference II	54
6.1	1. Recall: One-Sample Mean Problem	54
6.2	Model Diagnostics	55
6.2.1	Graphical Diagnostics for Normality	56
6.2.2	Formal Hypothesis Testing for Normality	57
6.3	Interpretation of Results	60
6.3.1	Point Estimation	60

6.3.2	Confidence Intervals	61
6.3.3	Hypothesis Testing	61
7	One Sample Nonparametric Test	63
7.1	Motivation	63
7.1.1	Key Characteristics of Nonparametric Tests	63
7.2	One Sample Nonparametric Test in SAS	64
7.3	Discussion: One-Sided vs Two-Sided Tests	67
7.3.1	Approximate One-Sided p-Value Calculation	67
7.3.2	Why Does This Work?	68
8	One Sample Proportion Test	70
8.1	Motivation	70
8.1.1	Sampling Distribution of the Sample Proportion	71
8.1.2	Confidence Interval for a Proportion	71
8.1.3	Standard Error of the Sample Proportion	72
8.1.4	Interpretation: Effect of Sample Size	72
8.2	Computation	73
8.3	Automating the Computations with PROC FREQ	74
8.4	Visualization of Binomial Proportion	77
8.4.1	Why visualization matters	77
8.4.2	Adding a reference line	77
8.4.3	Including sample size on the plot	78
8.4.4	Interpreting the plot	78
8.4.5	Practical visualization advice	78
8.4.6	Summary	79
9	SAS Macro Program in One Sample Variance Problem	80
9.1	Motivation	80
9.1.1	Examples	80
9.2	SAS Macro Program	82
9.3	How Macros Work	82
9.4	Designing Your Own Macros	83
9.4.1	Macros vs. Macro Variables	83
9.5	Think Globally and Locally: Scope of Macro Variables	84
9.5.1	Local Macro Variables	84
9.5.2	Global Macro Variables	84
9.5.3	Common Mistakes to Avoid	84
9.6	Writing SAS Macros with Car dataset	85
9.7	Revisit the One-Sample Variance Test	87
9.7.1	Using the VARTST Macro in SAS	90

10	χ^2 Goodness of Fit Test	93
10.1	Motivation: What Is a Goodness-of-Fit Test?	93
10.2	Example: Categorical Data (Dice Example)	94
10.3	Hypotheses in Probability Vector Form	94
10.4	One-Way Contingency Table	94
10.5	One-Way Contingency Table and Model Fit	95
10.6	Test Statistics	95
10.6.1	Deviance Test Statistic	96
10.7	How Do They Work?	96
10.8	Dice Rolls Example	97
10.8.1	Pearson chi-square statistic	97
10.8.2	Deviance (likelihood-ratio) statistic	98
10.9	Tomato Phenotypes Example	102
10.9.1	Null hypothesis	103
10.9.2	Expected frequencies	103
10.9.3	Goodness-of-fit results	103
11	Power Analysis with application in one sample t-test	109
11.1	1 Introduction	109
11.1.1	Key Components of Power Analysis	109
11.2	Motivation Examples	110
11.3	Before We Start the Analysis	111
11.4	Power Analysis in SAS	111
11.5	Discussion	120
11.6	Math behind the power analysis	121
11.7	A Visualization Example	122
11.7.1	Interpretation of the Power Visualization	123
12	Two Sample t-test for Independent and Paired Data	124
12.1	Inference Goals of the Two Sample t -Test	128
12.2	Sample SAS code	129
12.2.1	Two Independent Sample t -test	129
12.2.2	Paired Sample t -test	132
12.3	Statistical Assumptions of the Two Sample t -Test	133
12.4	Other Two-Sample Tests	134
12.4.1	Two-Proportion Z Test	134
12.4.2	Two-Sample Variance F Test	135
12.4.3	Summary	135

III Regression Analysis	137
13 Aanalysis of Variance	138
13.1 Introduction	138
13.2 From Two-Sample t-Test to ANOVA	138
13.3 ANOVA as a Generalization of the Two-Sample t-Test	139
13.4 Extending to Multiple Groups	139
13.5 Why ANOVA Works	140
13.6 What's Next	140
IV Hands on Class	141
14 Writing Macro in SAS	142
14.1 Guideline: How to Write SAS Macros	142
14.2 Practice Examples in this activity	143
14.2.1 Dataset for this exercise: Court Length Data	143
14.3 First SAS Macro for Reporting Mean and Standard Deviation	143
14.4 Second SAS Macro for creating confidence interval and p-value from a Z-test .	145
14.5 Convert Everything into a Macro	147
References	151

Preface

Description

This course covers programming using the SAS statistical software package, and it provides an introduction to data analysis stressing the implementation using SAS.

Topics include two main parts:

- 1) **SAS Programming:** data management and manipulation, basic procedures, macro programming;
- 2) **Data Analysis:** descriptive statistical analysis, one- and two-sample inference, basic categorical data analysis, regression analysis, and other selected topics.

Prerequisites

MATH 4544/6544 – Biostatistics, or equivalent.

Instructor

[Chi-Kuang Yeh](#), Assistant Professor in the Department of Mathematics and Statistics, Georgia State University.

- Office: Suite 1407, 25 Park Place.
- Email: cych@gsu.edu.

Office Hour

10:00–13:00 on Monday, or by appointment.

Grade Distribution

- Assignments: 60%
- Exam: 20%
- Project: 20%

Assignment

- ☒ A1, due on Jan 28, 2026
- ☒ A2, due on Feb 8, 2026
- ☒ A3, due on Feb 15, 2026
- ☒ A4, due on Feb 28, 2026
- ☐ A5, TBA

Midterm

- ☐ March 4, 2026

Topics and Corresponding Lectures

Those chapters are based on the lecture notes. This part will be updated frequently.

Status	Topic	Lecture
	Welcome and Overview	1
	Part 1: Basics	
	Basic SAS Operation	2
	SAS Syntax	3
	Import and Export Data	4
	Random Variable	5
	Part 2: Statistical Analysis	
	One Sample Problem as Example	
	Introduction to Statistical Inference I	6
	Introduction to Statistical Inference II	7
	One Sample Nonparametric Test	8
	One Sample Proportion Test	9
	Introduction to SAS Marco	10
	Chi-Square GoF test	11
	Power Analysis and Calculation (Recorded)	12

Status	Topic	Lecture
	Class Activity I	13
	Two Sample t -test for Independent and Paired Datasets	15
	Exam	
	Exam	14
	Part 3: Regression Analysis	
	Analysis of Variance	16
	Intro to Regression Analysis	17
	Spring Break	
	Regression with Interaction Effects	18
	Linear Mixed Effects Model I	19
	Linear Mixed Effects Model II	20

Recommended Textbooks

- [Statistics 480: Introduction to SAS](#), The Pennsylvania State University.
- [SAS Training](#), SAS Institute.
- [SAS Resources](#), University of California, Los Angeles.

Acknowledgments

Special thanks to [Li-Hsiang Lin](#) for providing the base materials given on this website.

Part I

Introduction

1 Introduction to Basic SAS Operation

Learning objective:

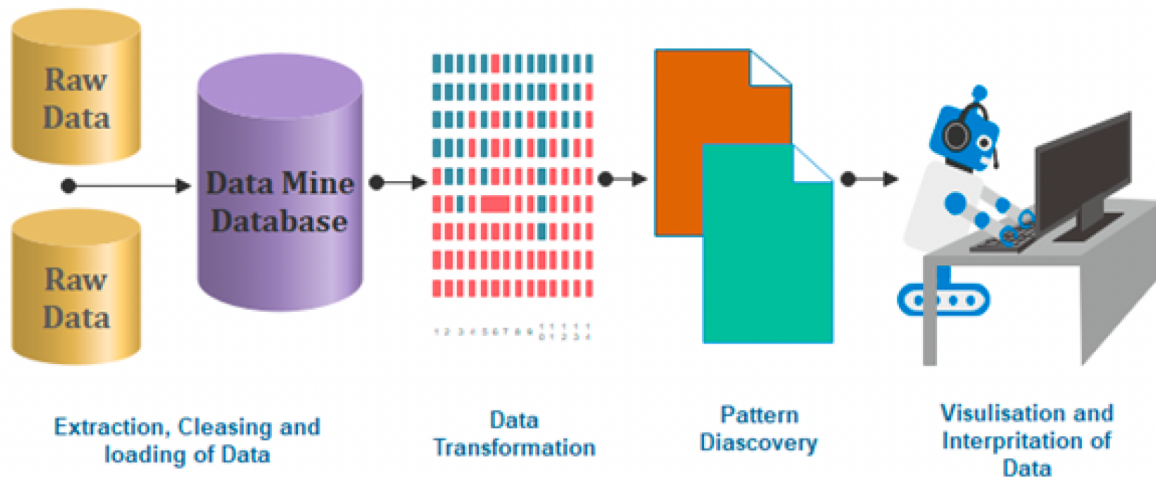
1. Familiarize ourselves with SAS windows (editor, log, output)
2. Create a dataset
3. Sorting Data (by 1 or more variables)
4. Obtain summary statistics of variables

1.1 Introduction to SAS

Q: What is SAS?

SAS (Statistical Analysis Software) is a prominent tool in the field of Data Analytics, offering a comprehensive suite for data manipulation, mining, management, and retrieval across various sources, coupled with robust statistical analysis capabilities. It excels in a range of functions including *data management, statistical analysis, report generation, business modelling, application development, and data warehousing*. SAS is user-friendly, featuring a point-and-click interface for those without technical expertise, while also providing deeper functionality through the SAS programming language. This software is instrumental in employing qualitative methods and processes that enhance employee productivity and business profitability.

Within SAS, data extraction and categorization into tables are pivotal for identifying and understanding data trends. This versatile suite supports advanced analytics, business intelligence, predictive analysis, and data management, facilitating effective operation in dynamic and competitive business environments. Additionally, SAS's platform-independent nature allows it to operate seamlessly across various operating systems, including Linux, Windows, Mac, and Ubuntu. SAS provides extensive support to programmatically transform and analyze data in the comparison of drag and drop interface of other Business Intelligence tools. It provides very fine control over data manipulation and analysis.



1.1.1 SAS Installization

Georgia State University (GSU) has purchased license, so we can access SAS University Edition for free!

To install SAS University Edition, choose from the following options:

- **Option 1:**

Download on your personal PC: Free SAS license available to GSU students, faculty, and staff via Technology Services (download required; check system requirements): Download from <https://technology.gsu.edu/technology-services/software-equipment/university-licensed-software/> (Need to log-in from your GSU Account)

SAS

For **students, faculty, and staff.**

LOG IN TO DOWNLOAD SAS

Log in to the university software download center with your **CampusID** and **CampusID Password**.

[DOWNLOAD ON WINDOWS](#)

[RENEW YOUR SAS LICENSE](#)

Get Help for the Installation from <<https://gsutech.service-now.com/sp>>

- **Option 2:**

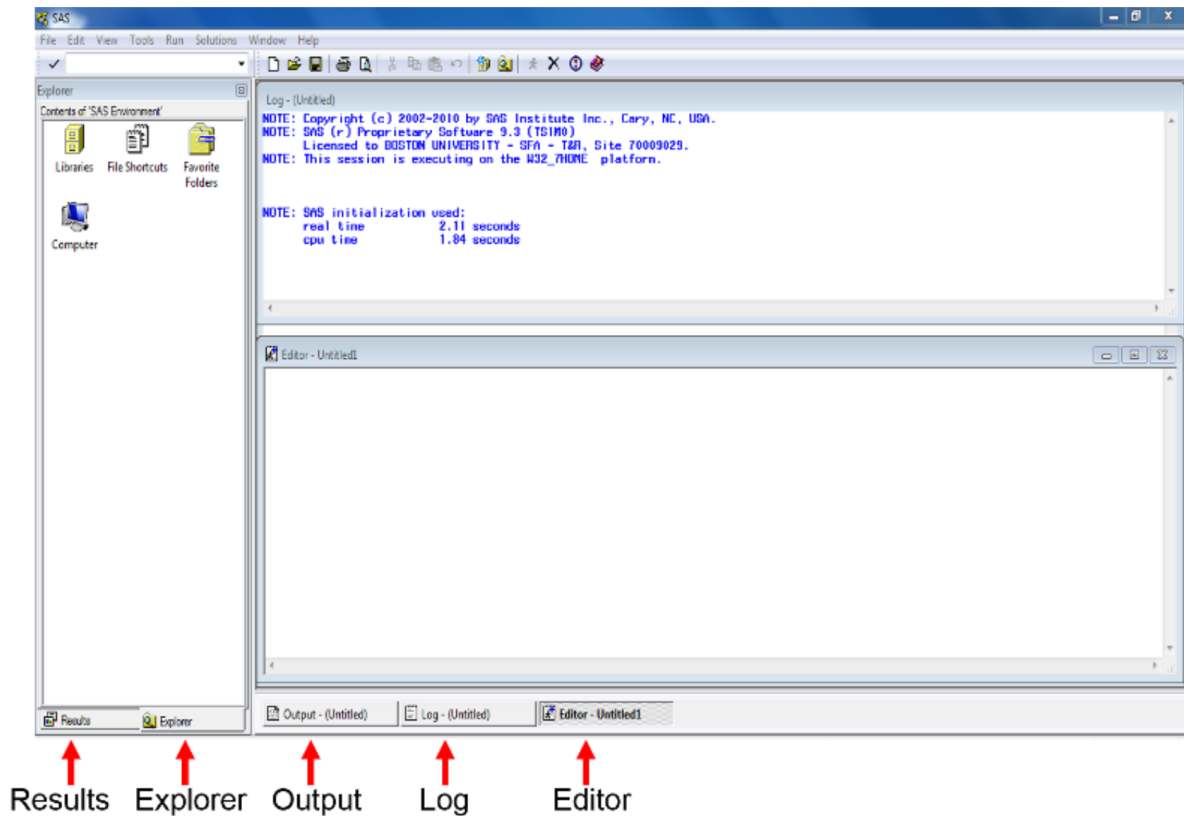
- On Campus Access: SAS can be found on all GSU Library PCs: Floors 1-4 (not available on Library Macs, because there is no Mac version of SAS)
- Graduate Biostatistics Computer Lab (SPH): 6th floor of the Urban Life building (swipe card access required)
- Common MILE Lab whose opening time is
 - * Monday & Wednesday: 9 – 18
 - * Tuesday & Thursday: 9 – 17
 - * Friday: 9 – 15
- **Option 3:**

Access via VLab, GSU's Remote Desktop Environment. Download and Connect to Cisco AnyConnect Client to connect to GSU's VPN (secureaccess.gsu.edu). Once connected to the VPN, login to VLab at: <https://vlab.gsu.edu/> to access SAS.
- **Option 4:**

Access via SAS OnDemand for Academics/SAS Studio. If you do not already have one, create a SAS profile at <https://welcome.oda.sas.com/> Then, sign in with credentials and click SAS®Studio to access the web-based SAS environment.

1.1.2 SAS Windows

Once SAS has started, the screen will look similar to the following: The main SAS window is divided into several sub-windows:



- The menu and toolbar along the top of the window
- The **explorer/results browser** along the left hand side, where you can a listing of the results of successful SAS program.
- The **log** to the top right. This gives you information about possible errors after you have run your SAS program.
- The **program editor** below the log on the bottom right, where you create your SAS program.
- The windows bar along the bottom for you to switch all windows.

The **Editor (Program Editor)** window is a text editor that facilitates writing SAS programs (code). The Log window displays system messages, errors, and resource usage and is thus used to review program statements. The Output window displays output from statistical procedures run within the SAS program; however this is no longer the default. In SAS 9.3 output is sent to the Results Viewer which opens automatically when you run a procedure that generates output. The Results window displays a map of the Output window, and is useful for navigating the results of complicated analyses. Finally, the Explorer window contains all of the data sets in the current SAS session.

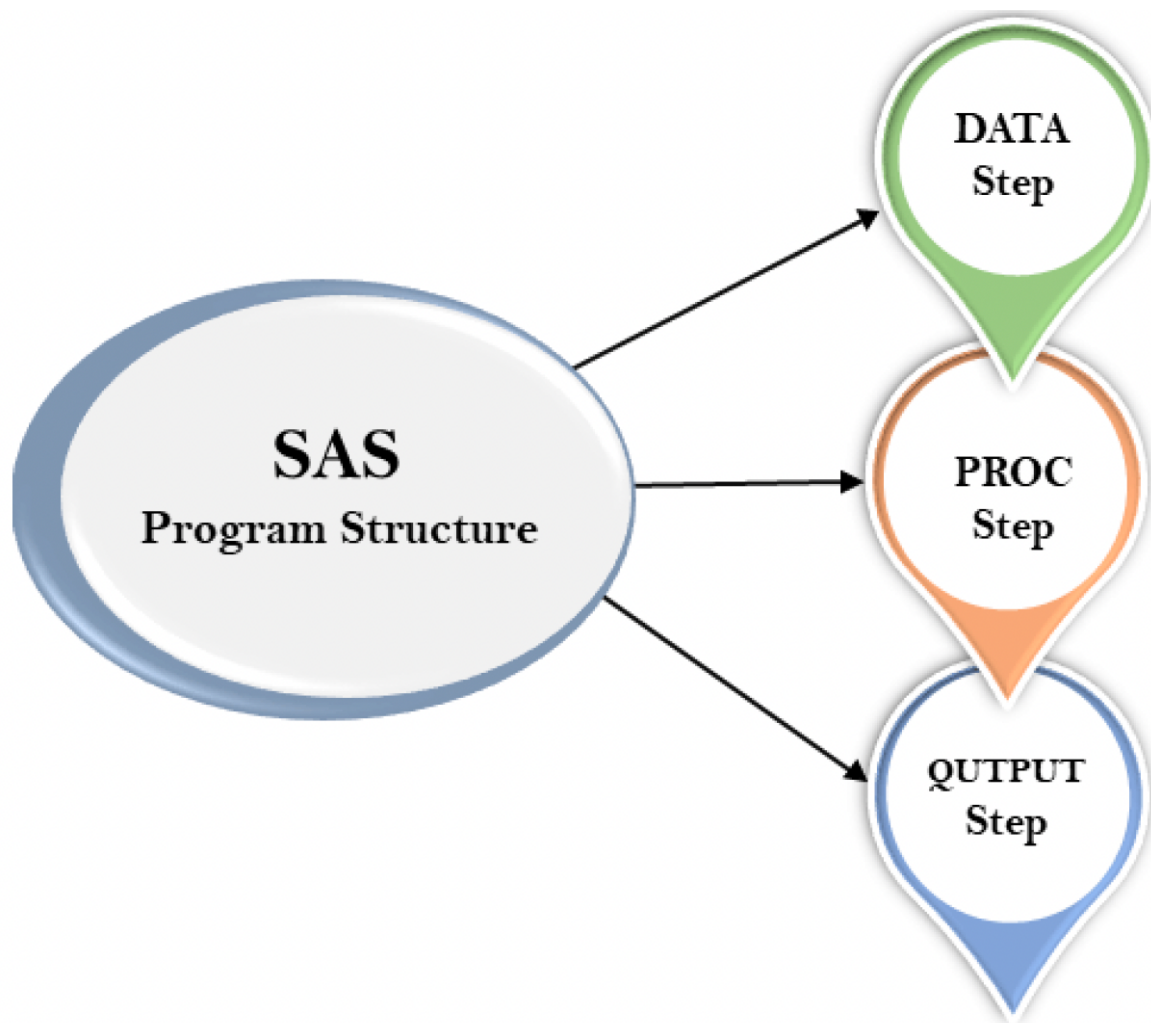
These windows can be moved or resized as desired. Only one SAS window is active at a time. The active window will have a shaded title bar at the top of the window, and a highlighted

windows bar at the bottom of the screen. In the above example, the Program Editor is the active window, with an "*Untitled*" program name. Note that the menu options for the SAS toolbar along the top of the screen depend on which window is currently active. (The active window can be changed by clicking on that window with the mouse, or by selecting the desired window from the Window menu.)

1.2 SAS Program

The programming structure of SAS consists of **three** significant steps:

- **DATA** step: create and modify a SAS data set for follow-up analysis
- **PROC** step: conduct data analysis
- **OUTPUT** step: show the analysis results



```
*Syntax of the SAS program;;  
DATA dataset name; /* Name of the data set. */  
INPUT var1,var2; /* Defines the variables in this data set. */  
NEW_VAR; /* Creates a new variable. */  
LABEL; /* Assign labels to variables. */  
DATALINES; /* Enters the data. */  
RUN;
```


1.3 SAS Dataset

SAS dataset is used to organize data values in a tabular form, i.e., in the form of rows for observations and columns for variables.

A SAS data set is a matrix whose each column is for each **variable** and whose each row for each **observation** (e.g., subject).

Data sets can be entered in the SAS programming code or can be read in from a variety of external sources, such as text files, csv files, and Microsoft Excel. In subsequent classes we will discuss reading in data sets from external files. Once a data set has been created, commands or procedures can operate on these data sets.

Other than these steps programming structure also includes data set, label, variables, values, and run.

1.4 SAS Examples

1.4.1 Creating a Dataset

Our first task in using SAS will be to create a small dataset and “print” that dataset to the output window. As we mentioned in previous paragraph SAS programs usually start with a DATA step where the dataset is created. Once the dataset is available, various procedures can be run on the dataset. The example below is written in the SAS Program window. The program creates a dataset called “People” with 3 variables (columns) which are ‘gender’, ‘height’, and ‘weight’ and 14 observations (rows). Note that the values of the variables on each line are separated by one or more blanks. A few other things that you should note:

- All SAS statements end with a semicolon (;)
- More than one SAS statement can be put on a line, or a SAS statement can continue across several lines, if every statement *ends with a semicolon*.
- Data listed as part of the program is also terminated with a semicolon. Data does not have to be entered in the program; it can also be read from files that are external to the SAS program (more on that next week)
- *gender* is a character variables as indicated by the \$, and height and weight are numeric variables.
- Once the dataset is created, various SAS procedures (called **PROC**s) can be used to analyze the data and present results. We will start with a listing of the data created with a procedure called **PROC PRINT**.

```
title1 'STAT 8678 Example 1';  
title2 'Your name';
```

```

DATA people;
INPUT gender $ height weight;

DATALINES;
m 63 125
m 76 195
f 62 109
m 75 186
f 67 115
f 60 120
m 75 205
m 71 185
m 63 140
f 59 135
f 65 125
m 68 167
m 72 220
f 66 155
;

PROC PRINT DATA=people;
RUN;

```

The LOG window gives information on the execution of the program. If your program did not execute properly you should examine the log for error messages that may explain the failure. The program would then be modified if necessary and rerun.

SAS creates a new window called the **Results Viewer** when the program is executed and produces output. This window is in HTML format and a new tab for the window is created below the left-hand windows.

1.4.2 Sorting Data

The data can be sorted (in our case by *gender*) using **PROC SORT** by adding the following lines to the program. We can then go ahead and print our new dataset sorted by gender with the proc print step.

Reminder

Any procedural step we do must begin with PROC and every line must end with a semicolon and the command run;

```
PROC SORT data = people;
BY gender;
RUN;

PROC PRINT data=people;
TITLE3 "Raw data sorted only by gender";
RUN;
```

Note

Any time we use PROC SORT our original dataset is sorted. SAS does not create a copy then sort!

1.5 Generating Summary Statistics

The last procedure to be executed in this exercise is **PROC UNIVARIATE**. This procedure will allow us to see summary statistics for any *quantitative variable*. The output from PROC UNIVARIATE will be important for the early part of this statistical methods class. It will provide measures of central tendency (mean, median, mode) and measures of dispersion (variance, standard deviation, range) as well as other basic statistics.

```
PROC UNIVARIATE DATA=people PLOT;
  BY gender;
  TITLE3 "Univariate procedure output done separately by gender";
  TITLE4 "The analysis was done for two quantitative variables";
  VAR HEIGHT WEIGHT;
RUN;
```

The code below applies the procedure only to the variable gender. It can be run on any quantitative variable and it could be run on several variables at the same time by listing several variables in the **VAR** statement, which would provide a separate analysis for each variable.

The results for PROC UNIVARIATE will be listed in the results viewer.

1.6 Other notes

Other Note

- SAS does not distinguish between upper-case letters or lower-case letters in the program, either can be used. However, it does distinguish between upper and lower case in datasets, so the character strings “Carol”, “carol” and “CAROL” would be considered different values of the variable “name” in the program above.
- Comments: Additionally, you may add comments anywhere in your program either by beginning the statement with an asterisk (*) and ending it with a semicolon (;) or by beginning with /* and ending with */. These comments may be thought of as marginal notes, and will show in the program editor and log, but not in the output window.

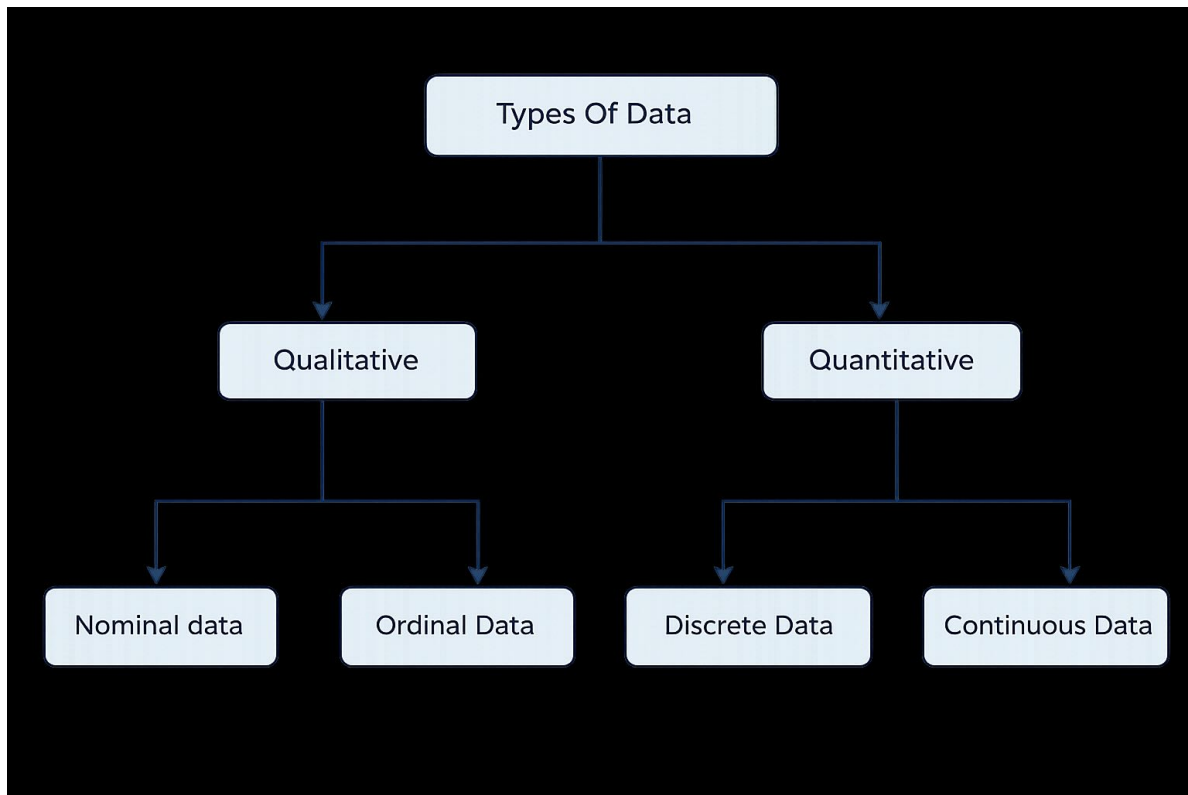
Shortcuts:

- F3 or the “running man”: submits/run your program;
- F4: recalls text once the program editor;
- F5: directs user to the program editor;
- F6: directs user to the log;
- F7: directs user to the output;
- Ctrl+E clears content in the current window.
- Also, text can be copied with Ctrl+C, cut with Ctrl+X, or pasted with Ctrl+V.

1.7 Data Type

We can classify variables into *quantitative* variables and *qualitative* variables:

- Qualitative variables yield non-numerical information. Qualitative variables are often referred to as categorical variables, such as blood type. Qualitative variables can be further classified as
 - A nominal variable is a qualitative variable where no ordering is possible or implied in the levels, such as gender.
 - An ordinal variable is a qualitative variable with an order implied in the levels, such as health (poor, reasonable, good, or excellent)
- Quantitative variables yield numerical measurements. Quantitative variables can be further classified as discrete or continuous.
 - A discrete variable can assume only a countable number of values, such as headache severity scores.
 - A continuous variable is one that can take any one of an uncountable number of values in an interval, such as weight.



Question:

What is the type of each variable in the following dataset?

- AGE: The respondent's age in years
- GENDER: The respondent's sex coded 1 for male and 2 for female
- HAPPY: The respondent's general happiness, coded:1 for "Not too happy"2 for "Pretty happy"3 for "Very happy"
- TVHOURS: The average number of hours the respondent watched TV during a day

Table 1.1: Survey Respondent Summary

Respondent	AGE	Gen	HAPPY	TVHOURS
1	41	1	2	0
2	25	2	1	0
3	43	1	2	4
4	38	1	2	2
5	53	2	3	2
6	43	2	2	6

7	56	2	2	2
---	----	---	---	---

Answer:

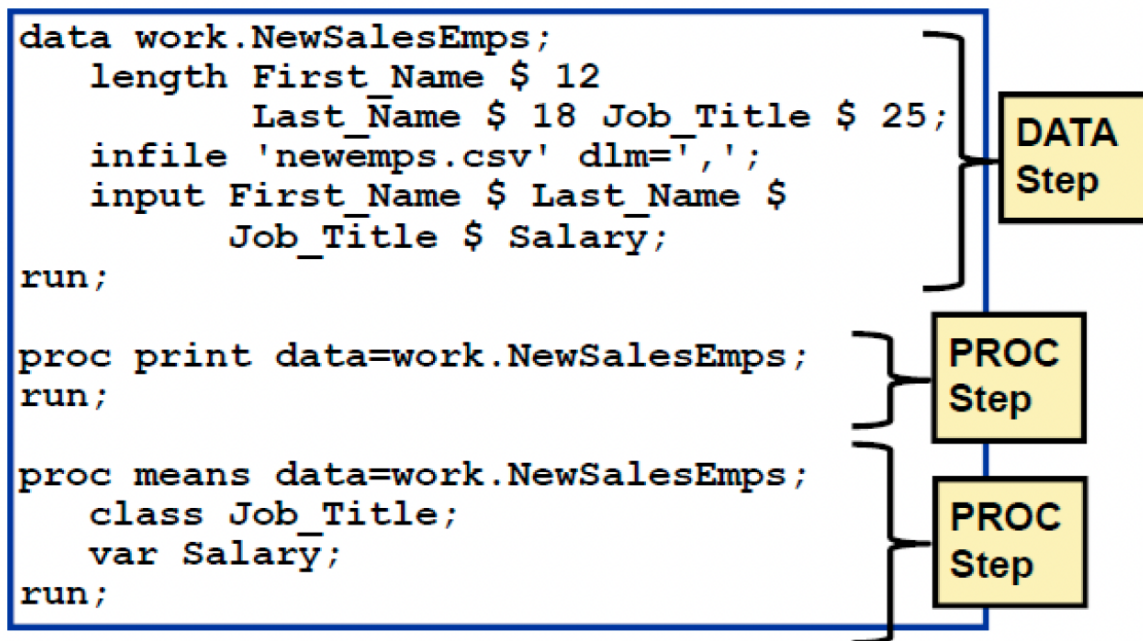
- Age: Continuous variable;
- SEX: qualitative;
- HAPPY: Discrete variable;
- TVHOURS: Continuous variable

2 Working with SAS Syntax

Learning objective:

1. Identify the characteristics of SAS statements.
2. Explain SAS syntax rules.
3. Insert SAS comments using two methods.
4. Identify SAS syntax errors.
5. Diagnose and correct a program with errors.
6. Save the corrected program

In general, writing a SAS program is to write a sequence of steps, and a step is a sequence of SAS statements:



How many statements are in the following step?

```

data work.NewSalesEmps;
    length First Name $ 12
           Last Name $ 18 Job Title $ 25;
    infile 'newemps.csv' dlm=' ',';
    input First Name $ Last Name $
          Job Title $ Salary;
run;

```

2.1 SAS Statments

SAS statements have these characteristics (Check the highlight context in the following examples):

- Usually begin with an identifying keyword.
- Always end with a semicolon.

```

data work.NewSalesEmps;
    length First Name $ 12
           Last Name $ 18 Job Title $ 25;
    infile 'newemps.csv' dlm=' ',';
    input First Name $ Last Name $
          Job Title $ Salary;
run;

proc print data=work.NewSalesEmps;
run;

proc means data=work.NewSalesEmps;
    class Job Title;
    var Salary;
run;

```


SAS programming statements are easier to read if you begin DATA, PROC, and RUN statements in column one and indent the other statements. This makes it more structured. Also, consistent spacing also makes a SAS program easier to read.

```
data work.NewSalesEmps;  
  length First_Name $ 12  
         Last_Name $ 18 Job_Title $ 25;  
  infile 'newemps.csv' dlm=',';  
  input First_Name $ Last_Name $  
        Job_Title $ Salary;  
run;  
  
proc print data=work.NewSalesEmps;  
run;  
  
proc means data=work.NewSalesEmps;  
  class Job_Title;  
  var Salary;  
run;
```

Conventional Formatting

Overall, we can type SAS statements in the following ways:

- One or more blanks can be used to separate words.
- They can begin and end in any column.
- A single statement can span multiple lines.
- Several statements can be on the same line.

```
data work.NewSalesEmps;
length First_Name $ 12
Last_Name $ 18 Job_Title $ 25;
infile 'newemps.csv' dlm=',';
input First_Name $ Last_Name $
Job_Title $ Salary;
run;
proc print data=work.NewSalesEmps; run;
proc means data =work.NewSalesEmps;
class Job_Title; var Salary;run;
```

Unconventional Formatting

(A)

```
data work.NewSalesEmps;
length First_Name $ 12
Last_Name $ 18 Job_Title $ 25;
infile 'newemps.csv' dlm=',';
input First_Name $ Last_Name $
Job_Title $ Salary;
run;
proc print data=work.NewSalesEmps; run;
proc means data =work.NewSalesEmps;
class Job_Title; var Salary;run;
```

Unconventional Formatting

(B)

```
data work.NewSalesEmps;
length First_Name $ 12
Last_Name $ 18 Job_Title $ 25;
infile 'newemps.csv' dlm=',';
input First_Name $ Last_Name $
Job_Title $ Salary;
run;
proc print data=work.NewSalesEmps; run;
proc means data =work.NewSalesEmps;
class Job_Title; var Salary;run;
```

Unconventional Formatting

(C)

```
data work.NewSalesEmps;
length First_Name $ 12
Last_Name $ 18 Job_Title $ 25;
infile 'newemps.csv' dlm=',';
input First_Name $ Last_Name $
Job_Title $ Salary;
run;
proc print data=work.NewSalesEmps; run;
proc means data =work.NewSalesEmps;
class Job_Title; var Salary;run;
```

Unconventional Formatting

(D)

```
data work.NewSalesEmps;
length First_Name $ 12
Last_Name $ 18 Job_Title $ 25;
infile 'newemps.csv' dlm=',';
input First_Name $ Last_Name $
Job_Title $ Salary;
run;
proc print data=work.NewSalesEmps; run;
proc means data =work.NewSalesEmps;
class Job_Title; var Salary;run;
```

Unconventional Formatting

(E)

Sometimes we may want to make comments/notes to easy remember our SAS statements or to let other people easily understand what our code want to say. These comments are text that SAS ignores during processing. You can use comments anywhere in a SAS program to document the purpose of the program, explain segments of the program, or mark SAS code as non-executing text.

There are two ways to insert comments in a SAS program:

1. Using an asterisk (*) to begin the comment and a semicolon (;) to end the comment.

```
* This is a comment in SAS;
```

2. Using slash-asterisk (/*) to begin the comment and asterisk-slash (*/) to end the comment.

```
/* This is a comment in SAS */
```

Avoid placing the /* comment symbols in columns 1 and 2. On some operating environments, SAS might interpret these symbols as a request to end the SAS job or session. An example is given below:

```

*-----*
|   This program creates and uses the   |
|   data set called work.NewSalesEmps.  |
*-----*;
data work.NewSalesEmps;
    length First_Name $ 12 Last_Name $ 18
           Job_Title $ 25;
    infile 'newemps.csv' dlm=',';
    input First_Name $ Last_Name $
           Job_Title $ Salary /*numeric*/;
run;
/*
proc print data=work.NewSalesEmps;
run;
*/
proc means data=work.NewSalesEmps;
    *class Job_Title;
    var Salary;
run;

```

2.2 Diagnosing and Correcting Syntax Errors

Syntax errors occur when program statements do not conform to the rules of the SAS language.

Examples of syntax errors:

- misspelled keywords
- unmatched quotation marks
- missing semicolons
- invalid options

When SAS encounters a syntax error, SAS prints a warning or an error message to the log; for example,

```
ERROR 22-322: Syntax error, expecting one of the following:
          a name, a quoted string, (, /, ;;, _DATA_, _LAST_,
          _NULL_.
```

When SAS encounters a syntax error, SAS underlines the error and the following information is written to the SAS log:

- the word **ERROR** or **WARNING**
- the location of the error + an explanation of the error

This program has three syntax errors. What are the errors?

```
daat work.NewSalesEmps;
  length First_Name $ 12
          Last_Name $ 18 Job_Title $ 25;
  infile 'newemps.csv' dlm=' ',';
  input First_Name $ Last_Name $
        Job_Title $ Salary;
run;

proc print data=work.NewSalesEmps
run;

proc means data=work.NewSalesEmps average max;
  class Job_Title;
  var Salary;
run;
```

2.3 Solution to the example

1. Exercise 1: 5
2. Exercise 2:

```
data work.NewSalesEmps;  
  length First_Name $ 12  
         Last_Name $ 18 Job_Title $ 25;  
  infile 'newemps.csv' dlm=',';  
  input First_Name $ Last_Name $  
        Job_Title $ Salary;  
run;  
  
proc print data=work.NewSalesEmps  
run;  
  
proc means data=work.NewSalesEmps average max;  
  class Job_Title;  
  var Salary;  
run;
```

3 Import and Export Dataset

Learning objective:

1. Import a csv file into SAS
2. Export a csv file from SAS after data process

One of the strengths of SAS as a data analysis tool is its ability to read data from many sources, subset or combine data sets, and modify the datasets to accomplish various tasks. The most common types of external data sets used in SAS are EXCEL files (XLS extent), comma separated value files (CSV extent) and various space separate text files (PRN or TXT extent). A CSV file is actually a text file and can be read in any text reader (NOTEPAD or WORDPAD in Windows). In fact, the SAS files themselves, as well as the LOG and the LST files produced by a SAS by a batch submit, are also simple text files.

Format	Description	File Extension
WK1	Lotus 1 spreadsheet	.WK1
WK3	Lotus 3 spreadsheet	.WK3
WK4	Lotus 4 spreadsheet	.WK4
EXCEL	Excel Version 4 or 5 spreadsheet	.XLS
EXCEL4	Excel Version 4 spreadsheet	.XLS
EXCEL5	Excel Version 5 spreadsheet	.XLS
EXCEL97	Excel 97 spreadsheet	.XLS
DLM	delimited file (default delimiter is a blank)	.*
TAB	delimited file (tab-delimited values)	.TXT
CSV	delimited file (comma-separated values)	.CSV

3.1 Reading from External Files

The **PROC IMPORT** statement is the best way to enter external data sets. The CSV file we will be using is called “grades.csv”. Download and save it in your favourite folder and mark the complete path to it. Then use the following code to import it, making sure you put the correct path on the **DATAFILE** argument.

```
PROC IMPORT OUT= GRADES_temp
  DATAFILE= "Put Your Path Here/grades_temp.csv" DBMS=CSV REPLACE;
  GETNAMES=YES;
  DATAROW=2;
RUN;
```

The **IMPORT** statement reads the dataset and stores it as the value designated by “OUT” in this case it will be saved as “Grades” in the library “Work”.

The **DBMS** statement defines the type of input SAS should be reading. The following table gives you all the possible choices. The **REPLACE** argument forces SAS to overwrite any older datasets with the same name.

The **GETNAMES** = YES or NO statement for spreadsheets and delimited external files, determines whether to generate SAS variable names from the column names in the input file’s first row of data. If you specify **GETNAMES** = NO or if the column names are not valid SAS names, PROC IMPORT uses the variable names VAR0, VAR1, VAR2, and so on. You may replace the equals sign with a blank.

The **DATAROW** argument tells SAS where to start reading for input data. In our case it is row 2 since row 1 is used for variable names.

We use the print procedure to see the dataset,

```
PROC PRINT DATA=GRADES_temp;
RUN
```

The first few rows are shown as follows

The SAS System										
Obs	Student	Quiz1	Quiz2	Quiz3	Quiz4	Quiz5	Quiz6	Midterm	EC	Gender
1	.	7	7	7	7	7	7	25	2	M
2	1	4.55	6.8	6	5.1	5.75	6.7	13	1	M
3	2	5	6.1	6.7	5.4	7	3.8	21.5	1	M
4	3	3.75	6.5	6.9	2.1	6.6	5.5	21	2	M
5	4	5.05	6.2	7	5.9	6.75	6	24.5	1	M
6	5	5.35	6.8	6.6	5.1	6.6	6.5	24	1	M
7	6	7	7	6.7	6.1	7	7	25	1	F

3.2 Export a File from SAS

After reading a dataset into SAS, we may need to conduct some initial step/analysis to re-organize dataset for doing further data analysis. In the *grade* example, the first row shows the maximum points available for each quiz. We need to remove this row so that our analysis is correct. This can be done by the following SAS code

```
DATA GRADES;  
  SET GRADES_temp NOBS=COUNT  
    IF _n_      <= 1 THEN DELETE;  
RUN;
```

Note: `GRADES_temp` is the name of the old dataset and *GRADES* is the name of the new dataset after the first row is deleted.

After re-organize the dataset, we may want to export the updated dataset for use in the future:

This can be done by

```
PROC EXPORT DATA= GRADES  
  OUTFILE= "Put Your Path Here/grade_v2.csv" DBMS=CSV REPLACE;  
  REPLACE;  
RUN;
```

We will talk about more reorganizing skills in SAS in the next topic.

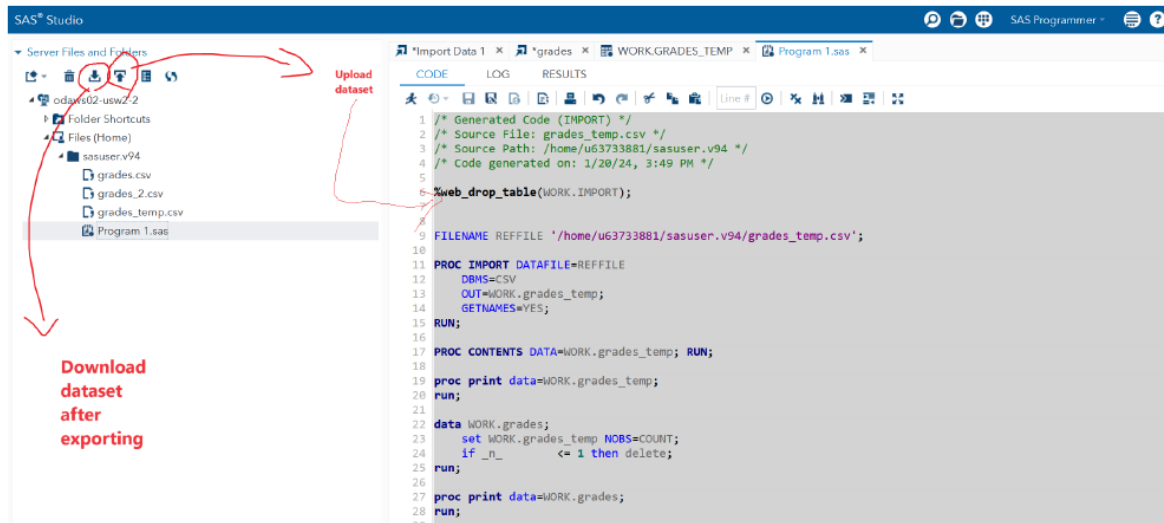
Which statement is true concerning the `DATALINES` statement based on reading the comment?

- The `DATALINES` statement is used when reading data located in a raw data file.
- The `DATALINES` statement is used when reading data located directly in the program.

3.3 Import & Export in SAS Virtual Studio

The key is

- Upload data before conducting “Import”
- Download data after conducting “Export”



A example given below will be demonstrated during the class.

```

/* Generated Code (IMPORT) */
/* Source File: grades_temp.csv */
/* Source Path: /home/u63733881/sasuser.v94 */
/* Code generated on: 1/20/24, 3:49 PM */

%web_drop_table(WORK.IMPORT);

FILENAME REFFILE '/home/u63733881/sasuser.v94/grades_temp.csv';

PROC IMPORT DATAFILE=REFFILE
    DBMS=CSV
    OUT=WORK.grades_temp;
    GETNAMES=YES;
RUN;

PROC CONTENTS DATA=WORK.grades_temp;
RUN;

PROC print data=WORK.grades_temp;
RUN;

DATA WORK.grades;
SET WORK.grades_temp NOBS=COUNT;
IF _n_ <= 1 THEN DELTE;
RUN;

```

```
PROC PRINT DATA=WORK.grades;  
RUN;  
  
PROC EXPORT data=WORK.grades  
  OUTFILE = "/home/u63733881/sasuser.v94/grades_2.csv"  
  DBMS = csv  
  REPLACE;  
RUN;
```

3.4 Solution to the example

Answer of the example: b

4 Random Variables

Learning Objectives

1. Distinguish between **discrete** and **continuous** random variables
2. Define random variables for real-world data problems
3. Identify appropriate probability distributions for common data types
4. Connect random variables to **data columns** used in SAS programs to represent a real world question

4.1 What Is a Random Variable?

A **random variable (RV)** is a numerical quantity whose value depends on the outcome of a random experiment.

We typically denote a random variable by an uppercase letter, such as X and its realized value by a lowercase letter, such as $X = x$. The random variable can be *continuous* or *discrete*.

In practice, we often observe multiple realizations, or running the random experiment multiple times, say n times or n realizations. We denote these realizations as:

$$X_1 = x_1, X_2 = x_2, \dots, X_n = x_n.$$

The number n is called the **sample size**.

Define whether the following random variables are discrete or continuous, and the possible values that X takes.

- Number of emails received by a server in one hour ($X = 0, 1, 2, \dots$: discrete)
- Time (in minutes) until a machine fails ($X = x \in [0, \infty)$: continuous)
- Total number of defects on a manufactured item ($X = 0, 1, 2, \dots$: discrete)
- Daily maximum temperature in Atlanta (in degrees Fahrenheit) ($X = x \in (-\infty, \infty)$: continuous)
- Whether a randomly selected loan defaults within one year ($X = 1$ if default, 0 otherwise: discrete)

Define whether the following random variables are discrete or continuous, and the possible values that X takes.

1. Number of transactions made by a customer in a day
2. Response time (in seconds) of a web service request
3. Count of hospital admissions in a city per week (discrete)
4. Proportion of time a system is idle during a day

4.2 Discrete vs Continuous Random Variables

4.2.1 Discrete Random Variables

A **discrete random variable** takes values in a **finite or countable set**.

Examples:

- Number of heads in three coin flips
- Number of students passing an exam
- Number of events occurring in a fixed time period

A discrete random variable is described by a **probability mass function (PMF)**:

Value of X	x_1	x_2	x_3	...	x_m
Probability	p_1	p_2	p_3	...	p_m

These probabilities satisfy:

- $0 \leq p_i \leq 1$,
- $\sum_{i=1}^m p_i = 1$.

We calculate the probability of events modelled by discrete random variables by summing up the probability p_i for the values x_i that make up the event.

Suppose the length X (in minutes) of an international phone call has distribution:

X	1	2	3	4
$P(X)$	0.2	0.5	0.2	0.1

Then, calculate the following probabilities

- a. $P(X \leq 2)$
- b. $P(X < 2)$
- c. $P(X > 1)$

4.3 Common Discrete Distributions

4.3.1 Bernoulli Distribution

Used for **binary outcomes**:

- success / failure
- yes / no
- 1 / 0

Notation: $X \sim \text{Ber}(p)$, where p is the probability of success.

Example:

Whether a patient has diabetes (1 = yes, 0 = no).

How to specify the probability p will be discussed in the following lecture. This is related to the procedure of statistical inference.

4.3.2 Poisson Distribution

Used to model **counts of events over time or space**.

Notation: $X \sim \text{Poi}(\lambda)$

where λ is the **mean rate**.

Examples:

- Number of trades per day
- Number of system failures per week
- Number of arrivals to a service queue

Similar to the probability p from the Bernoulli distribution, how to specify the rate λ will be discussed in the following lecture.

4.4 Continuous Random Variables

A **continuous random variable** can take **any value** x in an interval.

Examples:

- Height of individuals
- Time until failure of a component
- Test scores treated as continuous

Probabilities are defined using **density functions**, not point probabilities. Some common continuous distributions are as follows.

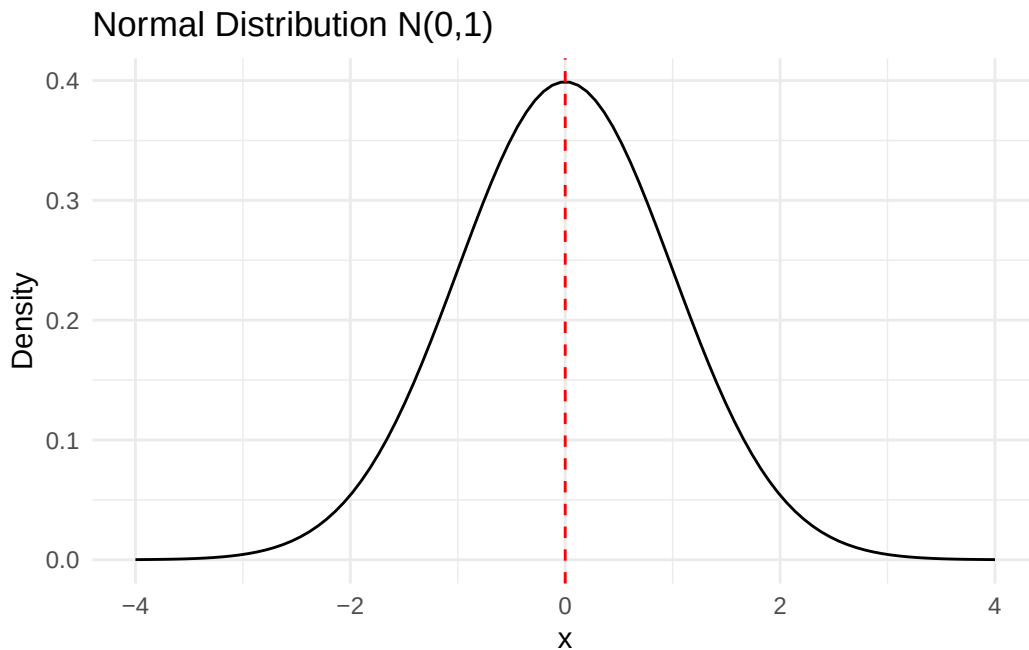
4.4.1 Normal Distribution

The normal distribution is also called the **Gaussian distribution**. It is characterized by two parameters: the mean μ and the standard deviation σ . It is unimodal and symmetric around the mean which is the centre of the mass. A continuous random variable X that has a normal distribution is said to be *normal* or *normally distributed*.

Notation: $X \sim N(\mu, \sigma^2)$. Sometimes the standard deviation may be used instead of the variance, which is the square of the standard deviation.

Properties of the normal distribution:

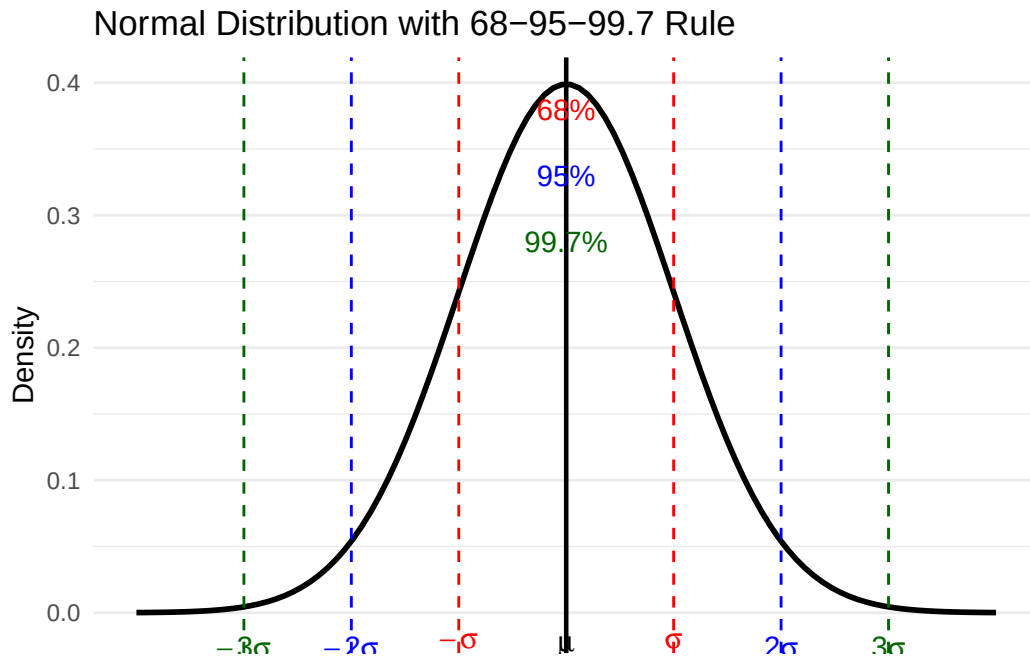
- mean: μ
- variance: σ^2



A distribution plot of the normal distribution with mean 0 and standard deviation 1 is shown above. It is often referred to as the *standard normal distribution*. In practice, we would like to “standardize” the data to have mean 0 and standard deviation 1 for the subsequent analysis.

Normal distribution plays an important role in statistical inference. One of the important properties is the 68-95-99.7 rule: - About 68% of the data falls within one standard deviation of

the mean - About 95% of the data falls within two standard deviations of the mean - About 99.7% of the data falls within three standard deviations of the mean



4.4.2 Exponential Distribution

The exponential distribution is another common distribution where a few outcomes are most likely with a rapidly decreasing probability for larger values. It is often used to model waiting times or lifetimes of objects. It is similar to the *geometric distribution* in the discrete case.

Two ways to specify the parameter.

1. $X \sim \text{Exp}(\lambda)$, where λ is the *rate* parameter.
2. $X \sim \text{Exp}(\beta)$, where β is the *scale* parameter, and $\beta = 1/\lambda$.

The notation is either $X \sim \text{Exp}(\lambda)$ or $X \sim \text{Exp}(\beta)$. But to be sure which parameterization is used, we need to check the definition of the distribution.

Please define adequate random variables for the following data example, and think about what distribution can be used to model the data.

- Suppose we want to know whether the rate of diabetes of a certain area is too high. The researchers randomly discuss with 10 individuals in a certain area to know whether they have diabetes. The data are collected as “yes” or “no” answers as follows

Individual	1	2	3	4	5	6	7	8	9	10
Outcome	1	0	0	0	0	1	0	0	0	0

- A company that manufactures light bulbs claims that a particular type of light bulb will last 850 hours on average. To justify the claim more scientifically, a researcher randomly selects 10 light bulbs of that type and measures the lifetimes (in hours) of the light bulbs as follows:

Light Bulb	1	2	3	4	5	6	7	8
	831	832	840	819	822	836	829	817

In the following classes we will assume the dataset we have can be well-modelled represented for the underlined studies. The data collection, however, is beyond the scope of the class. For those interested in data collection, please refer to the *survey sampling* or *experimental design* area.

4.5 Solution to the exercise

Answer of the exercise

Exercise 1

1. Number of transactions made by a customer in a day ($X = 0, 1, 2, \dots$: discrete)
2. Response time (in seconds) of a web service request ($X = x > 0$: continuous)
3. Count of hospital admissions in a city per week ($X = 0, 1, 2, \dots$: discrete)
4. Proportion of time a system is idle during a day ($X = x \in [0, 1]$: continuous)

Exercise 2:

- $P(X \leq 2) = 0.7$
- $P(X < 2) = 0.2$
- $P(X > 1) = 0.8$

Part II

Statistical Analysis

5 Introduction to Statistical Inference I

Learning Objectives

1. Be familiar with the difference about a probability problem and a statistical problem
2. Applied the one sample t-test to do inference for one sample mean problem

5.1 Probability versus Statistics

- In *probability*, we assume that random variables $X_1, \dots, X_n$ follow a distribution with **known parameters**. Under this model, we can calculate probabilities of events of interest.
- In *statistics*, although we still use a distribution to model $X_1, \dots, X_n$, the **parameters of the distribution are assumed to be unknown**. Our primary goal is to use observed data, the realization $X_1 = x_1, \dots, X_n = x_n$ —to make inference about these unknown parameters.

There are three common goals of statistical inference:

- **Point estimation**
- **Interval estimation**
- **Hypothesis testing**

Note

Note that there may be multiple valid methods for achieving each goal, even when working with the same dataset.

5.2 Example: One-Sample Mean Problem

The term *one sample* does **not** mean that there is only one observation. Instead, it means that there is **one population** under study.

In this problem, we are interested in making inference about the population mean, denoted by μ .

5.2.1 Problem Formulation

Suppose we want to conduct statistical inference on the mean length of a certain type of court case. Let

$$\mu = \text{the mean length of a certain type of court case.}$$

However, the true value of μ is unknown because we cannot observe all realizations of this process. As a result, statistical inference is required.

If μ were known, no inference would be necessary. What we can do in practice is to **collect data**. Suppose we randomly select 20 court cases of the same type from historical records and observe their case lengths (in days):

$$\begin{aligned} &43, 90, 84, 87, 116, 95, 86, 99, 93, 92, \\ &121, 71, 66, 98, 79, 102, 60, 112, 105, 98. \end{aligned}$$

From a statistical perspective, these observed values are treated as **realizations of random variables**. Specifically, we denote

$$X_1 = 43, X_2 = 90, \dots, X_{20} = 98.$$

To model the data, we assume that the random variables

$$X_1, X_2, \dots, X_{20}$$

are **independent and identically distributed** according to a normal distribution with mean μ and variance σ^2 , that is,

$$X_i \stackrel{\text{i.i.d.}}{\sim} N(\mu, \sigma^2), \quad i = 1, \dots, 20.$$

Here, both μ and σ^2 are unknown parameters. In this example, our primary interest lies in estimating the population mean μ , while the variance σ^2 is treated as a nuisance parameter.

We can use the **maximum likelihood estimator** (MLE) or the **method of moments estimator** (MME) to construct a point estimator of μ . Under this model, however, they coincide. The resulting estimator is the **sample mean**, denoted by

$$\hat{\mu} = \bar{X}_{20},$$

where

$$\bar{X}_{20} = \frac{1}{20} \sum_{i=1}^{20} X_i.$$

Under the assumed model, we may use either the **maximum likelihood estimator** or the **method of moments** to construct a point estimator for the population mean μ . In this model, both methods lead to the same estimator: the **sample mean**. We denote this estimator by

$$\hat{\mu} = \bar{X}_n.$$

Pay careful attention to the notation. We use a **capital letter** $\bar{X}_n$ to emphasize that the estimator itself is a **random variable**. Since the estimator is a function of the random variables $X_1, \dots, X_n$, it is also random and therefore follows a probability distribution. The probability distribution of an estimator is called its **sampling distribution**. We will return to the concept of sampling distributions later in the course.

5.2.2 Formal Definitions

To be more precise, we introduce the following definitions.

- A **statistic** is any function of the random variables $X_1, \dots, X_n$. Because it is a function of random variables, a statistic is itself a random variable and therefore has a probability distribution. If we use the parametric model to describe it, then there is a distribution that depends on the unknown parameters.
- A **(point) estimator** of a parameter θ , denoted by $\hat{\theta}$, is a statistic used to estimate θ . (Note, an estimator is a random variable because it is a statistics).
- A **(point) estimate** is the numerical value obtained by evaluating the estimator using the observed data $x_1, \dots, x_n$. In particular, we use **lowercase notation** to indicate an estimate. For example, $\bar{x}_n$ denotes the observed value of the estimator $\bar{X}_n$.

So far, we have constructed a point estimator (and hence a point estimate). To construct a confidence interval or perform hypothesis testing, we need to know the **sampling distribution** of the estimator $\bar{X}_n$.

Under the normal model with unknown variance, the sampling distribution can be expressed as

$$\frac{\bar{X} - \mu}{S/\sqrt{n}} \sim t_{n-1},$$

where

$$S = \sqrt{\frac{\sum_{i=1}^n (X_i - \bar{X})^2}{n-1}},$$

and t_{n-1} denotes the **Student's** t distribution with $n-1$ degrees of freedom.

5.2.3 Confidence Interval for μ

Based on this sampling distribution, a $(1-\alpha) \times 100\%$ confidence interval for μ is given by

$$\left[\bar{X} - t_{n-1, \alpha/2} \frac{S}{\sqrt{n}}, \bar{X} + t_{n-1, \alpha/2} \frac{S}{\sqrt{n}} \right].$$

5.2.4 Hypothesis Testing for μ

Using the same sampling distribution, we can conduct one of the following hypothesis tests:

- **Two-sided test**

$$H_0 : \mu = \mu_0 \quad \text{vs.} \quad H_1 : \mu \neq \mu_0$$

- **Left-tailed test**

$$H_0 : \mu = \mu_0 \quad \text{vs.} \quad H_1 : \mu < \mu_0$$

- **Right-tailed test**

$$H_0 : \mu = \mu_0 \quad \text{vs.} \quad H_1 : \mu > \mu_0$$

In some textbooks, equivalent hypotheses are written as

-

$$H_0 : \mu = \mu_0 \quad \text{vs.} \quad H_1 : \mu \neq \mu_0$$

-

$$H_0 : \mu \geq \mu_0 \quad \text{vs.} \quad H_1 : \mu < \mu_0$$

-

$$H_0 : \mu \leq \mu_0 \quad \text{vs.} \quad H_1 : \mu > \mu_0$$

Fortunately, **SAS** can perform point estimation, confidence interval construction, and hypothesis testing simultaneously, provided we clearly specify:

- **The statistical question**
(one-sample mean, one-sample variance, two-sample problem, etc.)
- **The inferential goal**
(point estimation, confidence interval, hypothesis testing)
- **The statistical model and method**
(for example, why we use the t distribution for the one-sample mean problem instead of the normal distribution or another alternative)

One of the main goals of this course is to help you build a mental “library” of statistical tools. Later, when you encounter a statistical question, you will be able to identify an appropriate method and then use SAS to obtain numerical results for interpretation.

5.3 Revisit the data example by using SAS

The court length data can be read by the following DATA step:

```
DATA TIME;
  INPUT TIME @@;
DATALINES;
43 90 84 87 116 95 86 99 93 92
121 71 66 98 79 102 60 112 105 98
;
RUN;
```

The only variable in the DATA set, *time*, is assumed to be normally distributed. The trailing at signs (@@) indicate that there is more than one observation on a line. The following statements invoke PROC TTEST for a one-sample t test:

```
PROC TTEST H0=80 PLOTS(SHOWH0) SIDES=2 ALPHA=0.1;
  VAR TIME;
RUN;
```

- THE VAR statement indicates that the *time* variable is being studied
- the H0= option specifies that the mean of the time variable should be compared to the null value rather than the default value of 0
- the PLOTS(SHOWH0) option requests that this null value be displayed on all relevant graphs
- the SIDE=2 option specifies that the focus of the research question, namely whether the mean count case length is not equal to 80 days, rather than less or greater than 80 days (in which case you would use the SIDE=L or SIDE=U options, respectively)

- the ALPHA=0.1 option requests 90% confidence interval rather than the default 95% confidence interval

The output is presented in the table below.

The TTEST Procedure

Variable: time

N	Mean	Std Dev	Std Err	Minimum	Maximum
20	89.8500	19.1456	4.2811	43.0000	121.0

Mean	90% CL Mean		Std Dev	90% CL Std Dev	
89.8500	82.4474	97.2526	19.1456	15.2002	26.2374

DF	t Value	Pr > t
19	2.30	0.0329

Note

Some SAS procedures produce graphs as automatically as they produce tables if the ODS GRAPHICS option is used. The graphs are integrated with tables in the ODS output.

```
ODS GRAPHICS ON;

PROC TTEST HO=80 PLOTS(SHOWHO) SIDES=U ALPHA=0.1;
  VAR TIME;
RUN;

ODS GRAPHICS OFF;
```

you will see the following results which include the same table you see in the last figure but with two more figures. We will talk those two Figures in the following classes, especially the QQ plot.

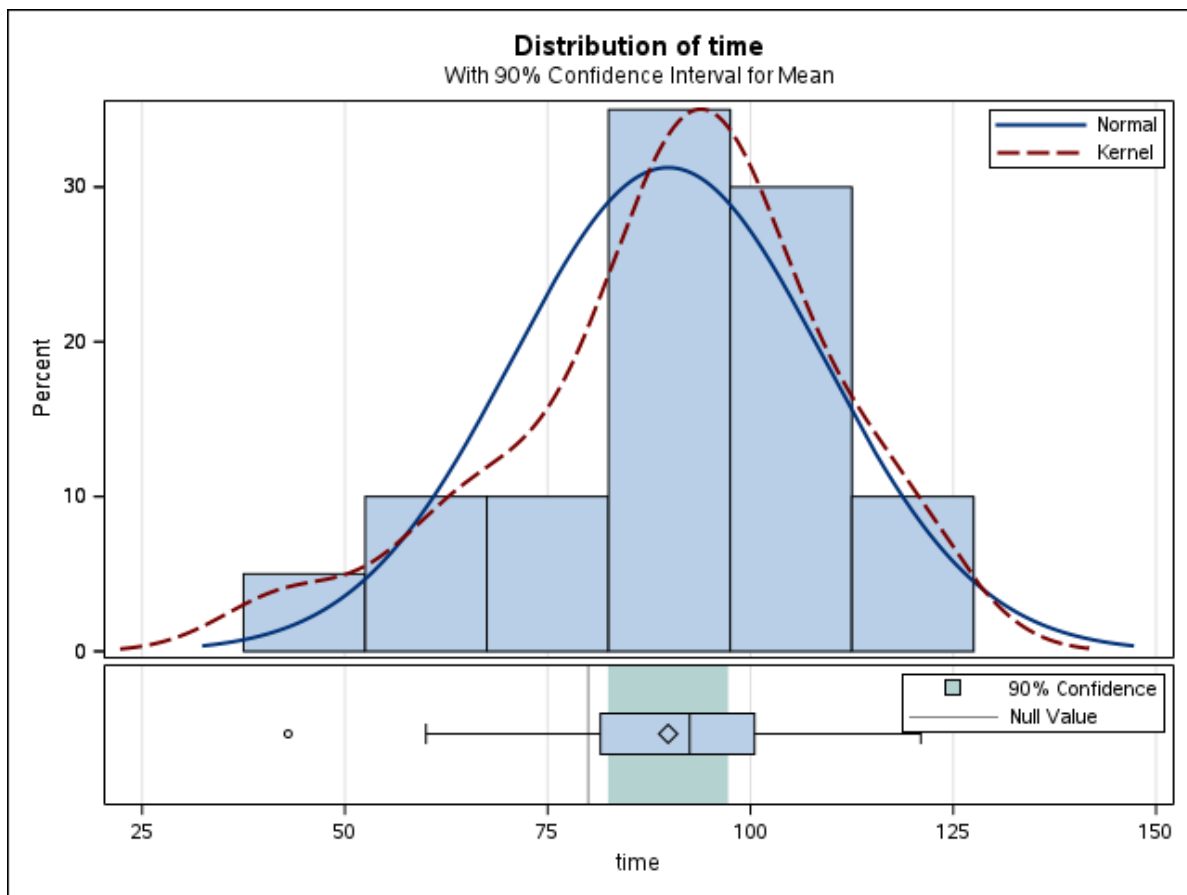
The TTEST Procedure

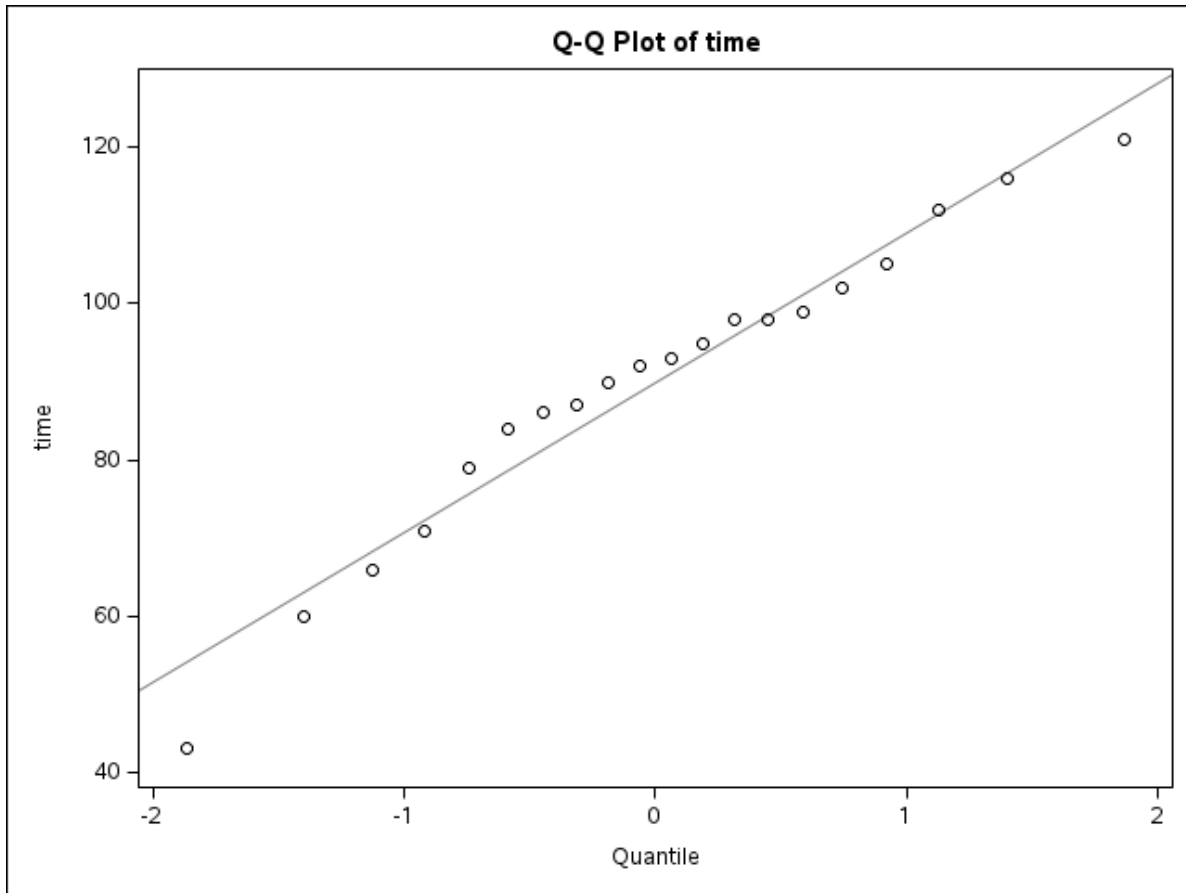
Variable: time

N	Mean	Std Dev	Std Err	Minimum	Maximum
20	89.8500	19.1456	4.2811	43.0000	121.0

Mean	90% CL Mean		Std Dev	90% CL Std Dev	
89.8500	82.4474	97.2526	19.1456	15.2002	26.2374

DF	t Value	Pr > t
19	2.30	0.0329





5.4 More about hypothesis and confidence intervals

5.4.1 How to construct a confidence interval?

Consider a **95% confidence interval** for the population mean μ . Under the normal approximation (or by the Central Limit Theorem), we have

$$P(-1.96 < Z < 1.96) \approx 0.95,$$

where Z is a standard normal random variable.

Equivalently, this can be written as

$$P\left(-1.96 < \frac{\bar{X} - \mu}{\sigma/\sqrt{n}} < 1.96\right) \approx 0.95.$$

5.4.2 Rearranging the Inequality

Rearranging the terms inside the probability statement yields

$$P\left(\bar{X} - 1.96 \frac{\sigma}{\sqrt{n}} < \mu < \bar{X} + 1.96 \frac{\sigma}{\sqrt{n}}\right) \approx 0.95.$$

5.4.3 Large-Sample Confidence Interval

Thus, a **large-sample 95% confidence interval** for μ is given by

$$\left[\bar{X} - 1.96 \frac{\sigma}{\sqrt{n}}, \bar{X} + 1.96 \frac{\sigma}{\sqrt{n}} \right].$$

More compactly, we often write this interval as

$$\bar{X} \pm 1.96 \frac{\sigma}{\sqrt{n}}.$$

5.5 3.2 Hypothesis Testing and Confidence Intervals Always Agree

5.5.1 Interpretation of the p -value

The p -value is the probability of observing data **at least as favorable to the alternative hypothesis** H_A as the data actually observed, **assuming the null hypothesis** H_0 is true.

In this example, the observed sample mean is either greater than 3.56 or less than 3.18, and the null hypothesis assumes the true population mean is $\mu = 3.37$.

5.5.2 Computing the p -value

We compute the p -value as

$$P(\bar{X} > 3.56 \text{ or } \bar{X} < 3.18 \mid \mu = 3.37).$$

This can be written as the sum of two tail probabilities:

$$P(\bar{X} > 3.56 \mid \mu = 3.37) + P(\bar{X} < 3.18 \mid \mu = 3.37).$$

Standardizing using the normal distribution yields

$$P\left(Z > \frac{3.56 - 3.37}{0.31/\sqrt{147}}\right) + P\left(Z < \frac{3.18 - 3.37}{0.31/\sqrt{147}}\right).$$

Evaluating the standardized values gives

$$P(Z > 7.43) + P(Z < -7.43).$$

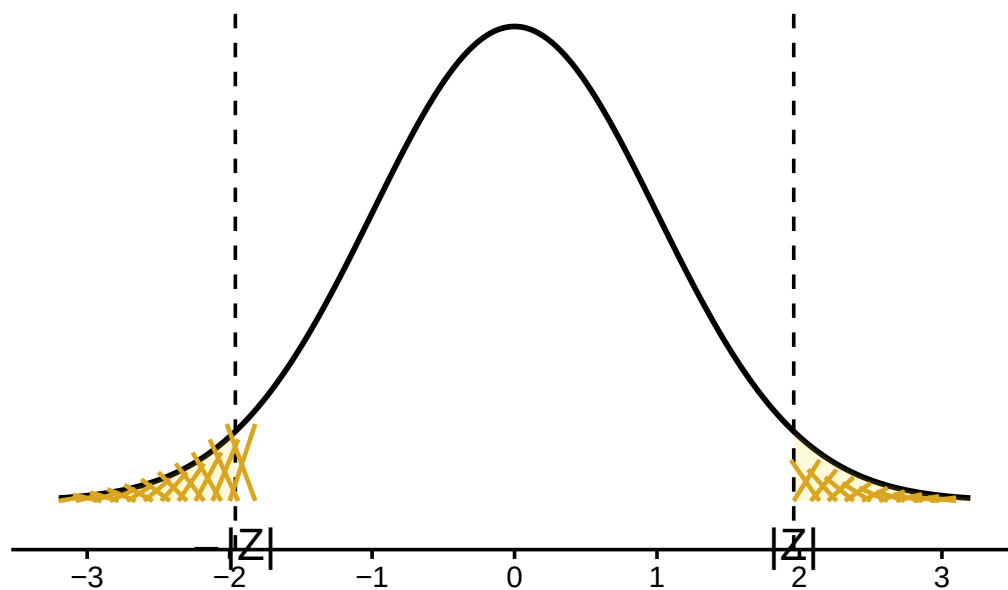
This probability is extremely small:

$$10^{-13} \approx 0.$$

5.5.3 Connection to Confidence Intervals

Because the p -value is essentially zero, we strongly reject H_0 at any reasonable significance level.

Equivalently, the hypothesized value $\mu = 3.37$ does **not** lie inside the corresponding confidence interval for μ .



This illustrates a key principle:

Hypothesis testing and confidence intervals always lead to the same conclusion when they are constructed at compatible significance levels.

6 Introduction to Statistical Inference II

Learning Objectives

1. Distinguish between a *probability problem* and a *statistical (inference) problem*
2. Apply the *one-sample t* test to inference for a population mean
3. Conduct *model diagnostics* for the normality assumption
4. Correctly *interpret point estimates, confidence intervals, and hypothesis tests*

6.1 1. Recall: One-Sample Mean Problem

In the previous class, we studied inference for a *population mean* and show how to run SAS to obtain the statistical inference result from one sample *t* test. The example for illustration is to study the mean length of a certain type of the court case, i.e.,

μ = the mean length of a certain type of court case.

The data for the study is

43, 90, 84, 87, 116, 95, 86, 99, 93, 92,
121, 71, 66, 98, 79, 102, 60, 112, 105, 98.

and we implement the statistical model:

$$X_i \stackrel{\text{i.i.d.}}{\sim} N(\mu, \sigma^2), \quad i = 1, \dots, 20.$$

With this set up, we can use SAS, such as through the following code

```
ODS GRAPHICS ON;  
  
PROC TTEST HO=80 PLOTS(SHOWHO) SIDES=U ALPHA=0.1;  
  VAR TIME;  
RUN;  
  
ODS GRAPHICS OFF;
```

and obtain the statistical results. The results let us know

- A point estimate of μ
- An interval estimate of μ
- A hypothesis testing under $H_0 : \mu = 80$ versus H_1 ,

from t -distribution for the one sample mean problem.

From the example, we may think the beginning of conducting statistical analysis procedure as

- Formulate the statistical problem
- Collection the adequate dataset and think the dataset as realization of some random variables
- Think some statistical models (i.e., assumptions) to be considered for the random variables. This step may include some [iv]
- preliminary data analysis

In this lecture, we will continue the remaining two steps:

- Model diagnostics
- Interpret the results

6.2 Model Diagnostics

In this step, we pause and think carefully about the assumptions underlying our analysis.

Question:

What statistical models are we implementing?

In this course, the key assumptions for the one-sample mean problem are:

- **A normality assumption**
- **An independence assumption**

The independence assumption is primarily determined by *how the data are collected*. Once the data have been sampled, this assumption is generally *not testable* from the data alone.

We therefore assume that the sampling process was conducted correctly and focus our attention on checking the *normality assumption*.

There are two broad classes of methods:

- *Graphical (visual) diagnostics*
- *Formal hypothesis testing methods*

6.2.1 Graphical Diagnostics for Normality

Common graphical tools include:

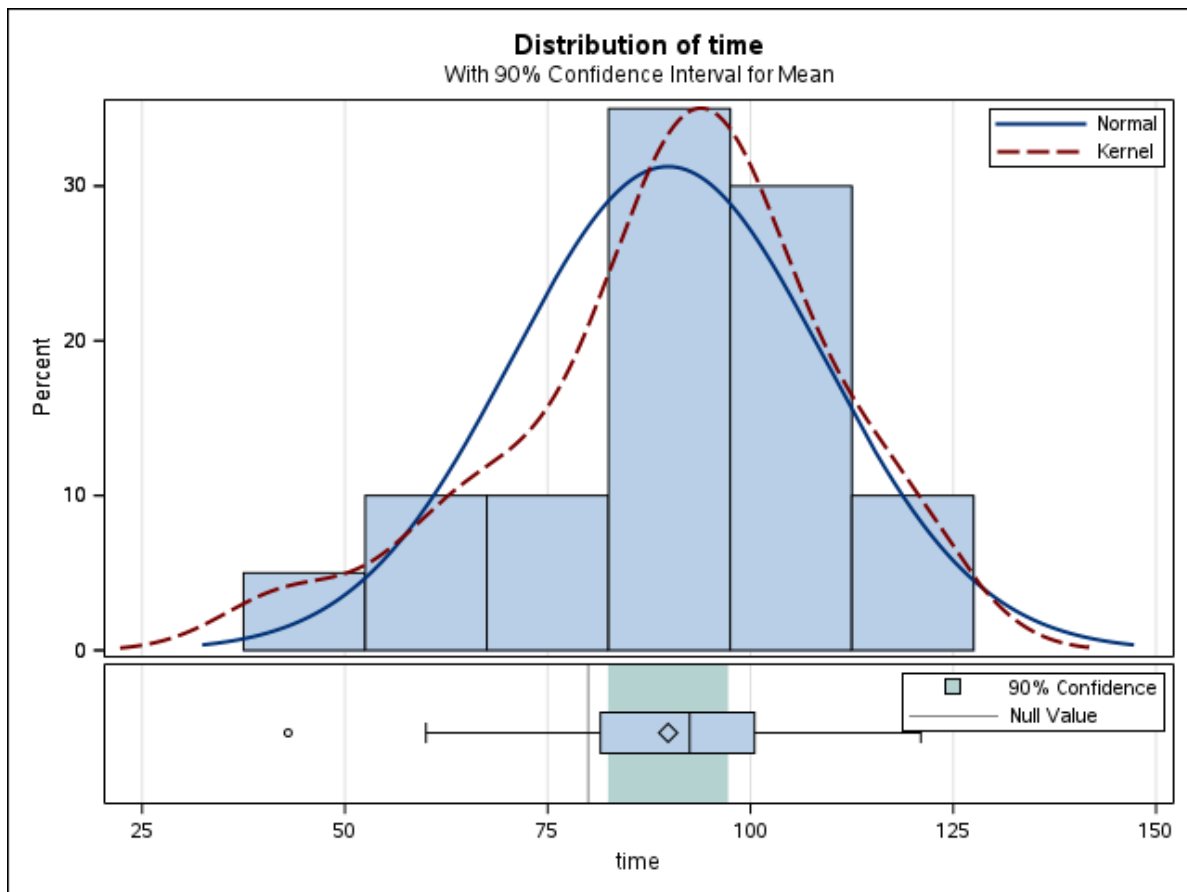
- **Histogram and Density Plots:**

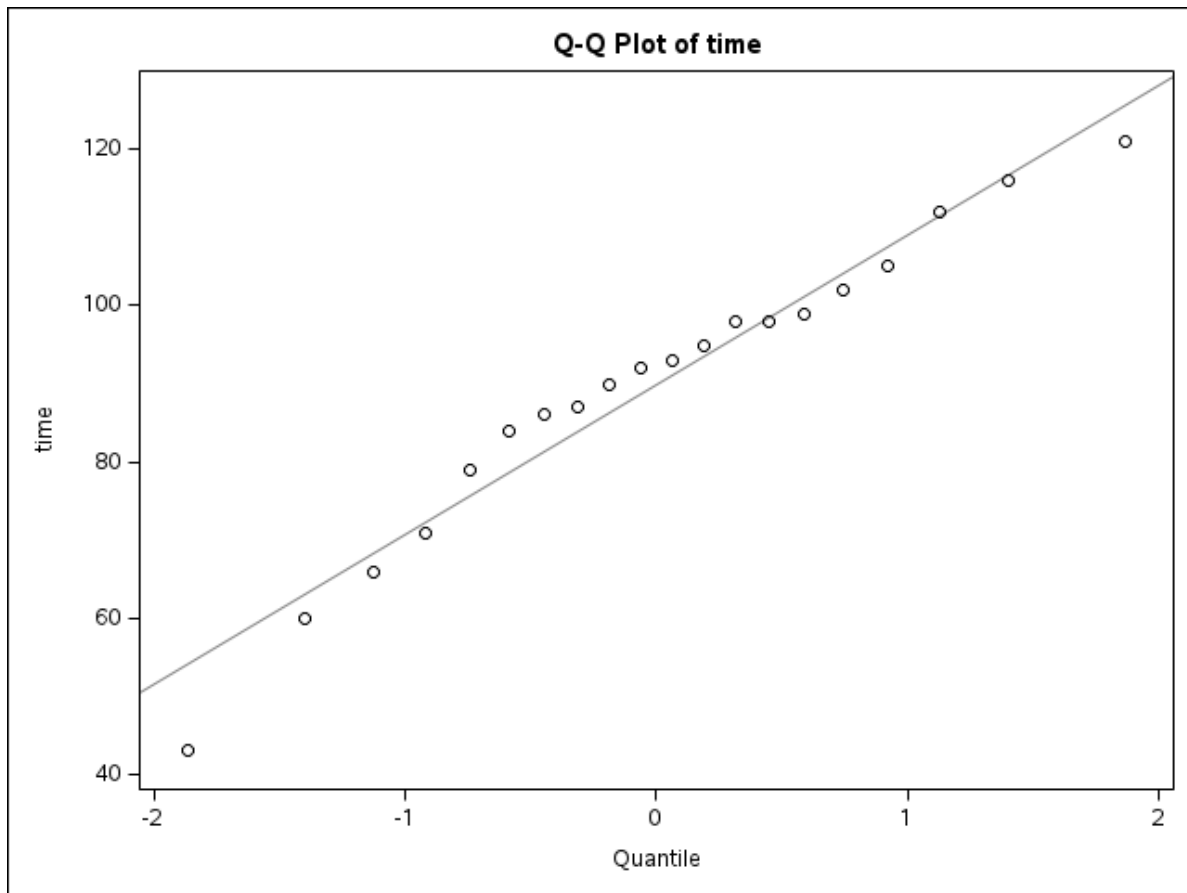
A kernel density estimate (KDE) provides a smooth estimate of the underlying probability density function, analogous to a histogram but without binning. KDEs represent the data using a continuous curve and are particularly useful for visualizing distributional shape.

- **Q–Q Plot (Quantile–Quantile Plot)**

A Q–Q plot compares *empirical quantiles* of the observed data with *theoretical quantiles* from a normal distribution.

If the normality assumption is reasonable, the points should fall approximately along a straight line.





6.2.2 Formal Hypothesis Testing for Normality

In addition to graphical methods, we can also assess the normality assumption using *formal numerical tests*.

One commonly used method is the **Kolmogorov–Smirnov (K–S) test**, which evaluates whether a sample plausibly comes from a specified distribution, such as the normal distribution.

The K–S test is widely used because many statistical procedures rely on the assumption that the data are normally distributed. When this assumption is violated, standard inference procedures may no longer be valid. The following step-by-step example demonstrates how to perform a Kolmogorov–Smirnov test for normality using **SAS**.

Step 1: Create the dataset

```
DATA time;  
  INPUT time @@;
```

```
DATALINES;  
43 90 84 87 116 95 86 99 93 92  
121 71 66 98 79 102 60 112 105 98  
;  
RUN;
```

Step 2: Perform the normality test

```
/* Perform Kolmogorov-Smirnov test */  
PROC UNIVARIATE DATA=time;  
    HISTOGRAM time / NORMAL(mu=est sigma=est);  
RUN;
```

The UNIVARIATE Procedure

Fitted Normal Distribution for time

Parameters for Normal Distribution		
Parameter	Symbol	Estimate
Mean	Mu	89.85
Std Dev	Sigma	19.14563

Goodness-of-Fit Tests for Normal Distribution				
Test	Statistic		p Value	
Kolmogorov-Smirnov	D	0.12997262	Pr > D	>0.150
Cramer-von Mises	W-Sq	0.05410070	Pr > W-Sq	>0.250
Anderson-Darling	A-Sq	0.31058808	Pr > A-Sq	>0.250

Quantiles for Normal Distribution		
Percent	Quantile	
	Observed	Estimated
1.0	43.0000	45.3106
5.0	51.5000	58.3582
10.0	63.0000	65.3139
25.0	81.5000	76.9365
50.0	92.5000	89.8500
75.0	100.5000	102.7635
90.0	114.0000	114.3861
95.0	118.5000	121.3418
99.0	121.0000	134.3894

i Remarks

Graphical diagnostics are informal but highly informative.

They allow us to detect skewness, heavy tails, and outliers that may invalidate normal-based inference.

Formal hypothesis tests for normality will be introduced next, but should always be interpreted in conjunction with these visual tools.

i Important Note

For the best practice:

Never rely on a single normality test. Always combine numerical tests with visual inspection.

6.3 Interpretation of Results

6.3.1 Point Estimation

Point estimation is the process of using sample data to compute a single numerical value that estimates an unknown population parameter, such as the population mean.

When we report a point estimate, it is desirable to understand whether the estimator has good statistical properties. In particular, we often examine whether a point estimator is:

- **Consistent:** As the sample size increases, the estimator becomes closer to the true parameter value.
- **Unbiased:** The expected value of the estimator equals the true population parameter. For example, the sample mean is an unbiased estimator of the population mean.
- **Efficient (or best unbiased):** Among all unbiased and consistent estimators, it has the smallest variance, meaning the estimator varies less from sample to sample.

How to rigorously verify these properties is a major topic in theoretical statistics courses. In practice, when these properties are unknown or difficult to assess, the safest interpretation of a point estimate is simply to report it with its associated unit. For example, using the court length data, we may state:

A reasonable estimate for the average court length is approximately 89.85 days.

6.3.2 Confidence Intervals

A confidence interval (CI) provides a range of plausible values for an unknown population parameter. A common—but informal—interpretation of a 95% confidence interval is that we are “95% confident” the true parameter lies within the interval. While this interpretation is widely used, the **strictly correct interpretation** is based on repeated sampling:

If the same study were repeated infinitely many times, and a 95% confidence interval were constructed each time, then approximately 95% of those intervals would contain the true parameter value.

As an example, suppose the 90% confidence interval for the population mean court length is [15.2, 26.2]. This means that, under the confidence interval procedure used, intervals constructed in this way would contain the true mean in 90% of repeated samples.

Factors Affecting Confidence Interval Width

The width of a confidence interval depends on several factors:

- **Sample size**
Larger samples typically produce narrower (more precise) confidence intervals.
- **Variability of the outcome**
For continuous outcomes, higher variability (larger standard deviation) leads to wider intervals.
- **Outcome type**
 - For binary outcomes, precision depends on the event probability.
 - For time-to-event outcomes, precision depends on the number of observed events.

All of these factors influence the standard error of the estimator, which directly determines the width of the confidence interval.

6.3.3 Hypothesis Testing

Hypothesis testing evaluates whether observed data are consistent with a specified statistical assumption, typically called the **null hypothesis**.

The result of a hypothesis test allows us to decide whether the assumption is supported by the data or whether there is sufficient evidence to reject it. The strength of this evidence is quantified by the *p-value*.

A **p-value** is defined as:

The probability of observing data at least as extreme as the data actually observed, assuming the null hypothesis is true.

For example, if a hypothesis test yields a p-value of 0.01, this means that—if the null hypothesis were true—there would be only a 1% chance of observing data this extreme.

A small p-value provides evidence against the null hypothesis, while a large p-value indicates that the data are consistent with it.

If the observed data are very unlikely to occur under the conditions described by the null hypothesis, then the null hypothesis is unlikely to be true. In such cases, we reject the null hypothesis in favor of the alternative hypothesis, and the result is said to be **statistically significant**.

A commonly used decision rule is based on a significance level of 0.05. If the p-value is less than or equal to 0.05, this is typically taken as evidence against the null hypothesis, and we reject it in favor of the alternative hypothesis. However, a p-value cannot be used to prove that a hypothesis is true. Instead, it quantifies the strength of evidence against the null hypothesis.

Consider the court length data. Suppose we conduct a hypothesis test with

$$H_0 : \mu = 80 \quad \text{versus} \quad H_1 : \mu > 80,$$

and obtain a p-value of 0.0164.

If we choose a significance level of 0.05, then since $0.0164 < 0.05$,

we reject the null hypothesis. At the 5% significance level, there is strong statistical evidence that the population mean court length is **not equal to 80**.

When the p-value Is Large

If the p-value is greater than 0.05, we **do not reject** the null hypothesis. This does **not** mean that the null hypothesis is true. Rather, it means that the data do not provide strong enough evidence to conclude that the population mean court length differs from 80.

In hypothesis testing, failing to reject the null hypothesis should be interpreted as *insufficient evidence*, not as confirmation of the null hypothesis.

7 One Sample Nonparametric Test

Learning Objectives

1. Be familiar with the difference about parameter tests and nonparameteric tests
2. Applied the one sample non-parametricc test to do inference for one sample mean problem

7.1 Motivation

A **parametric test** specifies certain assumptions about the distribution of responses in the population from which the sample is drawn. The validity and interpretability of parametric test results depend critically on whether these assumptions are satisfied.

In contrast, a **nonparametric test** is based on a model that imposes only very general conditions and does **not** assume a specific parametric form for the population distribution. For this reason, nonparametric tests are often referred to as **distribution-free tests**.

Although nonparametric methods are not completely assumption-free, the assumptions they require are generally **fewer and weaker** than those of parametric tests.

7.1.1 Key Characteristics of Nonparametric Tests

Nonparametric test statistics typically rely on simple features of the data, such as:

- Signs of measurements
- Ranks or orderings
- Category or frequency counts

As a result:

- Linear transformations (stretching or compressing the scale) do **not** affect the test statistic.
- The null distribution of the test statistic can often be derived **without specifying the population distribution**.

Nonparametric tests therefore avoid assumptions such as:

- Normality
- Homogeneity of variance

Moreover, nonparametric methods usually compare **medians rather than means**, which makes them less sensitive to outliers.

Advantages of Nonparametric Tests

- Applicable to **all measurement scales**
- Particularly useful when the **sample size is very small**, unless the distribution is known
- Easier to learn and compute
- Require **fewer assumptions**
- More **robust** due to weaker modeling assumptions
- Do not require explicit population parameters
- In some cases, results can be **as exact** as parametric procedures

Disadvantages of Nonparametric Tests

- Often **less powerful** than parametric tests when parametric assumptions hold
- Provide less information about population parameters
- Interpretation is usually framed in terms of **location or rank**, not means
- Some procedures do not extend easily to complex models

Summary

- Parametric tests rely on strong distributional assumptions but can be powerful and informative.
- Nonparametric tests trade efficiency for **robustness and flexibility**.
- In practice, nonparametric methods serve as valuable alternatives when assumptions are questionable or sample sizes are limited.

In the next section, we will study **specific one-sample nonparametric procedures** and implement them in **SAS**.

7.2 One Sample Nonparametric Test in SAS

Base SAS provides two commonly used **one-sample nonparametric tests** through the PROC UNIVARIATE procedure:

- **Sign test**
- **Wilcoxon signed-rank test**

Both tests are designed for situations in which we want to make inference about the **location** of a population, typically interpreted as the **median** rather than the mean.

Suppose we are interested in testing whether the **median resting pulse rate of marathon runners** differs from a specified value. If the normality assumption required for a one-sample *t*-test is questionable, nonparametric alternatives provide a robust solution.

By default, both tests examine the hypothesis that the median of the population from which the sample is drawn is equal to a specific value, which is zero by default. However, we note that

- **Wilcoxon signed-rank test**
 - Assumes the population distribution is **symmetric**
 - Generally more powerful when the symmetry assumption holds
- **Sign test**
 - Does **not** require symmetry
 - Uses only the **sign** of deviations from the hypothesized median
 - More robust but typically less powerful

Both tests can also be extended to **paired (related) samples**, which will be discussed later when we cover comparisons of two related samples.

Both the sign test and the Wilcoxon signed-rank test are **automatically available** in PROC UNIVARIATE.

```
/* Syntax: PROC UNIVARIATE */
PROC UNIVARIATE <options>;
    BY <variables>;
    CDFPLOT <variables> </ options>;
RUN;

CLASS variable-1 <(v-options)> <variable-2 <(v-options)>></ KEYLEVEL=value1 (value1 value2
)>;FREQ variable;HISTOGRAM <variables> </ options>;ID variables;INSET keyword-list </ option
<OUT=SAS-data-set> <keyword1=names ...keywordk=names> <percentile-options>;PPLOT <variables>
</ options>;PROBPLOT <variables> </ options>;QQPLOT <variables> </ options>;VAR variables;WE
variable;
```

- The PROC UNIVARIATE statement invokes the **UNIVARIATE procedure**, which provides detailed descriptive statistics, distributional summaries, and diagnostic plots for numerical variables.
- The VAR Statement
 - Specifies the **numeric variables** to be analyzed.
 - **Required** if the OUTPUT statement is used.

- If omitted, **all numeric variables** in the data set are analyzed.
- The PLOT statement (CDFPLOT, HISTOGRAM, PPLOT, PROBPLOT, and QQPLOT) create graphical displays
- the INSET statement enhances these displays by adding a table of summary statistics directly on the graph.

You can specify one or more of each of the plot statements, the INSET statement, and the OUTPUT statement. If you use a VAR statement, the variables listed in a plot statement must be a subset of the variables listed in the VAR statement.

You can specify a BY statement to obtain separate analyses for each BY group. The FREQ statement specifies a variable whose values provide the frequency for each observation. The ID statement specifies one or more variables to identify the extreme observations. The WEIGHT statement specifies a variable whose values are used to weight certain statistics.

You can use a CLASS statement to specify one or two variables that group the data into classification levels. The analysis is carried out for each combination of levels in the input data set, or within each BY group if you also specify a BY statement. You can use the CLASS statement with plot statements to create comparative displays, in which each cell contains a plot for one combination of classification levels.

We revisit the **court length example** to demonstrate how one-sample **nonparametric tests** can be used as an alternative or complement to the one-sample *t*-test.

Suppose that, for some reason, we believe the *t*-test may not be an appropriate choice—perhaps due to concerns about normality—or we simply wish to **double-check our conclusions** using nonparametric methods. In this case, we can apply the nonparametric procedures available in PROC UNIVARIATE.

Step 1: Input the Data

We begin by entering the court length data into SAS.

```
DATA time;
    INPUT time @@;
    DATALINES;
43 90 84 87 116 95 86 99 93 92
121 71 66 98 79 102 60 112 105 98
;
RUN;
```

Step 2: Perform a One-Sample Nonparametric Test

To test whether the population median court length differs from 80 days, we use PROC UNIVARIATE with the MU0= option.

```
/* Perform one-sample nonparametric test */  
PROC UNIVARIATE DATA=time MU0=80;  
    VAR time;  
RUN;
```

Step 3: Interpret the Output

From the output, SAS provides: + Test statistics + p-values for both nonparametric tests

These results allow us to assess whether there is statistical evidence that the population median court length differs from 80 days, without relying on the normality assumption required by the t-test.

7.3 Discussion: One-Sided vs Two-Sided Tests

Question:

Is this a *one-sided* or a *two-sided* test?

In this example, **SAS reports only two-sided p-values** by default for the nonparametric tests in PROC UNIVARIATE. If a **one-sided test** is desired, SAS does not directly provide the result.

However, a **simple (though not exact) workaround** can be used to approximate the one-sided p-value from the two-sided p-value.

7.3.1 Approximate One-Sided p-Value Calculation

1. Let

$$p^* = \frac{\text{two-sided p-value}}{2}.$$

2. Then proceed according to the alternative hypothesis:

7.3.1.1 Case 1: Right-sided test

Testing

$$H_1 : \mu > 80 \quad (\text{or } \mu > \mu_0)$$

- If the sample mean $\bar{x} > \mu_0$, then

$$\text{one-sided p-value} = p^*.$$

- If the sample mean $\bar{x} < \mu_0$, then

$$\text{one-sided p-value} = 1 - p^*.$$

7.3.1.2 Case 2: Left-sided test

Testing

$$H_1 : \mu < 80 \quad (\text{or } \mu < \mu_0)$$

- If the sample mean $\bar{x} < \mu_0$, then

$$\text{one-sided p-value} = p^*.$$

- If the sample mean $\bar{x} > \mu_0$, then

$$\text{one-sided p-value} = 1 - p^*.$$

7.3.2 Why Does This Work?

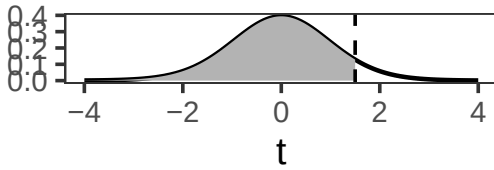
A two-sided p-value measures evidence in **both directions** away from the null hypothesis. Dividing it by two isolates the probability mass in **one tail** of the sampling distribution.

However, this adjustment is only valid when: - The test statistic is symmetric under the null hypothesis - The observed statistic is in the direction specified by the alternative

Because these conditions are not always guaranteed for nonparametric tests, this method should be viewed as an **approximation**, not an exact one-sided test.

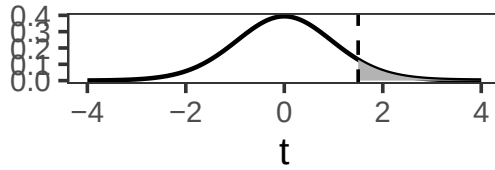
$$H_a : \mu < \mu_0$$

$$\text{p-value} = P(T \leq t)$$



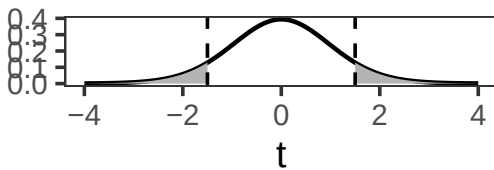
$$H_a : \mu > \mu_0$$

$$\text{p-value} = P(T \geq t)$$



$$H_a : \mu \neq \mu_0$$

$$\text{p-value} = 2P(T \geq |t|)$$



Key takeaway:

SAS nonparametric procedures default to two-sided inference. While approximate one-sided p-values can be obtained manually, interpretation should be done with care.

8 One Sample Proportion Test

Learning Objectives

1. Apply the binomial test to conduct statistical inference for a population proportion.

8.1 Motivation

Suppose we observe a **categorical variable with two levels**, such as

- SMOKES = 1: smoker
- SMOKES = 2: non-smoker

In this setting, we are often interested in making inference about the **population proportion** of interest, such as the proportion of people who smoke.

We denote this population proportion by p .

Common inferential goals include:

- Estimating the population proportion p
- Constructing a confidence interval for p
- Performing a hypothesis test for p

Specifically, we may test

$$H_0 : p = p_0,$$

$$H_1 : \begin{cases} p \neq p_0, & \text{(two-sided),} \\ p < p_0, & \text{(left-sided),} \\ p > p_0, & \text{(right-sided).} \end{cases}$$

8.1.1 Sampling Distribution of the Sample Proportion

Let $\hat{p}$ denote the **sample proportion**, computed as

$$\hat{p} = \frac{X}{n},$$

where

- X is the number of successes,
- n is the sample size.

When the sample size is sufficiently large (typically $n \geq 30$), the sampling distribution of $\hat{p}$ is approximately normal:

$$\hat{p} \sim N\left(p, \frac{p(1-p)}{n}\right).$$

This normal approximation forms the basis for both **confidence intervals** and **hypothesis testing** for a population proportion.

8.1.2 Confidence Interval for a Proportion

A $(1 - \alpha) \times 100\%$ confidence interval for p is given by

$$\left[\hat{p} - Z_{\alpha/2} \sqrt{\frac{\hat{p}(1-\hat{p})}{n}}, \quad \hat{p} + Z_{\alpha/2} \sqrt{\frac{\hat{p}(1-\hat{p})}{n}} \right].$$

Here, $Z_{\alpha/2}$ is the standard normal quantile satisfying

$$P(Z \geq Z_{\alpha/2}) = \alpha/2, \quad Z \sim N(0, 1).$$

Moreover, this normal approximation of $\hat{p}$ is also used to perform hypothesis testing for p .

8.1.3 Standard Error of the Sample Proportion

The **standard error** of $\hat{p}$ is

$$\text{SD}(\hat{p}) = \sqrt{\frac{\hat{p}(1 - \hat{p})}{n}}.$$

This expression highlights an important principle:

The uncertainty in an estimate decreases as the sample size increases.

8.1.4 Interpretation: Effect of Sample Size

Consider two examples where the estimated proportion is the same.

A small college has 8 physics majors, and 5 of them graduate within four years.

$$\hat{p} = \frac{5}{8} = 0.625$$

The standard error is

$$\text{SE}(\hat{p}) = \sqrt{\frac{0.6 \times 0.4}{8}} \approx 0.17.$$

An English department graduates 50 out of 80 students.

$$\hat{p} = \frac{50}{80} = 0.625$$

The standard error is

$$\text{SE}(\hat{p}) = \sqrt{\frac{0.6 \times 0.4}{80}} \approx 0.05.$$

Does this make sense?

i Note

In practice, when the normal approximation may not be reliable (e.g., small n), we will instead rely on **exact binomial methods**, which do not require large-sample assumptions.

8.2 Computation

Suppose a college has six majors, labelled A, B, C, D, E, and F. For each major, we observe:

- + the number of students who graduated within four years (Grads), and
- + the total number of students in that major (Total).

```
DATA Grads;  
  INPUT Major $ Grads Total @@;  
  DATALINES;  
A 10 22 B 10 32 C 17 25  
D 4 7 E 8 14 F 16 28  
;  
RUN;
```

Obs	Major	Grads	Total
1	A	10	22
2	B	10	32
3	C	17	25
4	D	4	7
5	E	8	14
6	F	16	28

It is easy to write a short DATA step to compute the empirical proportions and a 95% confidence interval for each major.

```
DATA GradRate;  
  SET Grads;  
  
  /* Empirical proportion */  
  p = Grads / Total;
```

```

/* Standard error under normal approximation */
StdErr = SQRT(p * (1 - p) / Total);

/* 95% Wald confidence interval */
z = QUANTILE("normal", 1 - 0.05/2);

LCL = MAX(0, p - z * StdErr); /* Lower bound */
UCL = MIN(1, p + z * StdErr); /* Upper bound */

LABEL p    = "Proportion"
      LCL = "Lower 95% CI"
      UCL = "Upper 95% CI";

RUN;

```

Major	Grads	Total	Proportion	Lower 95% CL	Upper 95% CL
A	10	22	0.45455	0.24648	0.66261
B	10	32	0.31250	0.15190	0.47310
C	17	25	0.68000	0.49714	0.86286
D	4	7	0.57143	0.20483	0.93803
E	8	14	0.57143	0.31220	0.83065
F	16	28	0.57143	0.38813	0.75473

The output shows that although majors D, E, and F have the same four-year graduation rate (57%), the estimate for the D group, which has only seven students, has twice as much variability as the estimate for the F group, which has four times as many students.

8.3 Automating the Computations with PROC FREQ

In the previous section, we computed the Wald confidence interval for each major using a DATA step. That approach is straightforward, but it becomes inconvenient if we want other binomial confidence intervals (e.g., exact/Clopper–Pearson, Wilson, etc.). A convenient alternative is to use PROC FREQ with the BINOMIAL option. However, PROC FREQ expects the data in an

event / nonevent format (a binary outcome with a frequency count), rather than the events / trials format (Grads, Total) we currently have.

So we first convert each major into two rows: + Graduated="Yes" with Count = Grads + Graduated="No" with Count = Total - Grads

Step 1: Convert events/trials into event/nonevent format

```
/*-----
  Convert Event/Trials format (Grads, Total)
  into Event/Nonevent format (Graduated, Count)
-----*/

data GradFreq;
  set Grads;

  Graduated = "Yes";
  Count = Grads;
  output;

  Graduated = "No";
  Count = Total - Grads;
  output;
run;

proc print data=GradFreq;
run;
```

Obs	Major	Grads	Total	Graduated	Count
1	A	10	22	Yes	10
2	A	10	22	No	12
3	B	10	32	Yes	10
4	B	10	32	No	22
5	C	17	25	Yes	17
6	C	17	25	No	8
7	D	4	7	Yes	4
8	D	4	7	No	3
9	E	8	14	Yes	8
10	E	8	14	No	6
11	F	16	28	Yes	16
12	F	16	28	No	12

Step 2: Run PROC FREQ to compute binomial CIs by major

```
/*-----  
Use PROC FREQ to analyze each major separately and compute  
binomial confidence intervals for Pr(Graduated="Yes")  
-----*/  
proc freq data=GradFreq noprint;  
  by notsorted Major;  
  
  tables Graduated / binomial(level="Yes" CL=wald);  
  weight Count;  
  
  output out=FreqOut binomial;  
run;
```

Step 3: Print a clean summary table

```
proc print data=FreqOut noobs label;  
  var Major N _BIN_ L_BIN U_BIN;  
  
  label _BIN_ = "Proportion"  
        L_BIN = "Lower 95% CI"  
        U_BIN = "Upper 95% CI";  
run;
```

Major	Number of Subjects	Proportion	Lower 95% CI	Upper 95% CI
A	22	0.45455	0.24648	0.66261
B	32	0.31250	0.15190	0.47310
C	25	0.68000	0.49714	0.86286
D	7	0.57143	0.20483	0.93803
E	14	0.57143	0.31220	0.83065
F	28	0.57143	0.38813	0.75473

8.4 Visualization of Binomial Proportion

It is often helpful to visualize estimated proportions together with their confidence intervals. When plotting several proportions on the same graph, it is a good idea to **sort the groups** in a meaningful way—most commonly by the estimated proportion itself. If two groups have the same estimated proportion, the sample size can be used as a tie-breaker.

8.4.1 Why visualization matters

A single number rarely tells the whole story. Plotting proportions with confidence intervals allows us to:

- Compare graduation rates across majors at a glance
- Assess uncertainty in each estimate
- See how sample size affects precision

Groups with **smaller sample sizes** tend to have **wider confidence intervals**, reflecting greater uncertainty.

8.4.2 Adding a reference line

It is often informative to add a **reference line** representing the **overall proportion**, regardless of group membership.

For the graduation data in this example, the overall proportion of students who graduate in four years is

$$\hat{p}_{\text{overall}} = \frac{65}{128} \approx 0.5078.$$

This reference line helps contextualize each major's graduation rate relative to the overall average.

Tip:

You can obtain the overall proportion in SAS by repeating the PROC FREQ analysis **without** a BY statement.

8.4.3 Including sample size on the plot

Because uncertainty depends strongly on sample size, it is good practice to display the **number of observations per group** directly on the graph.

In SAS, this can be done using the `YAXISTABLE` statement, which adds a table aligned with the y-axis showing the sample size for each group.

8.4.4 Interpreting the plot

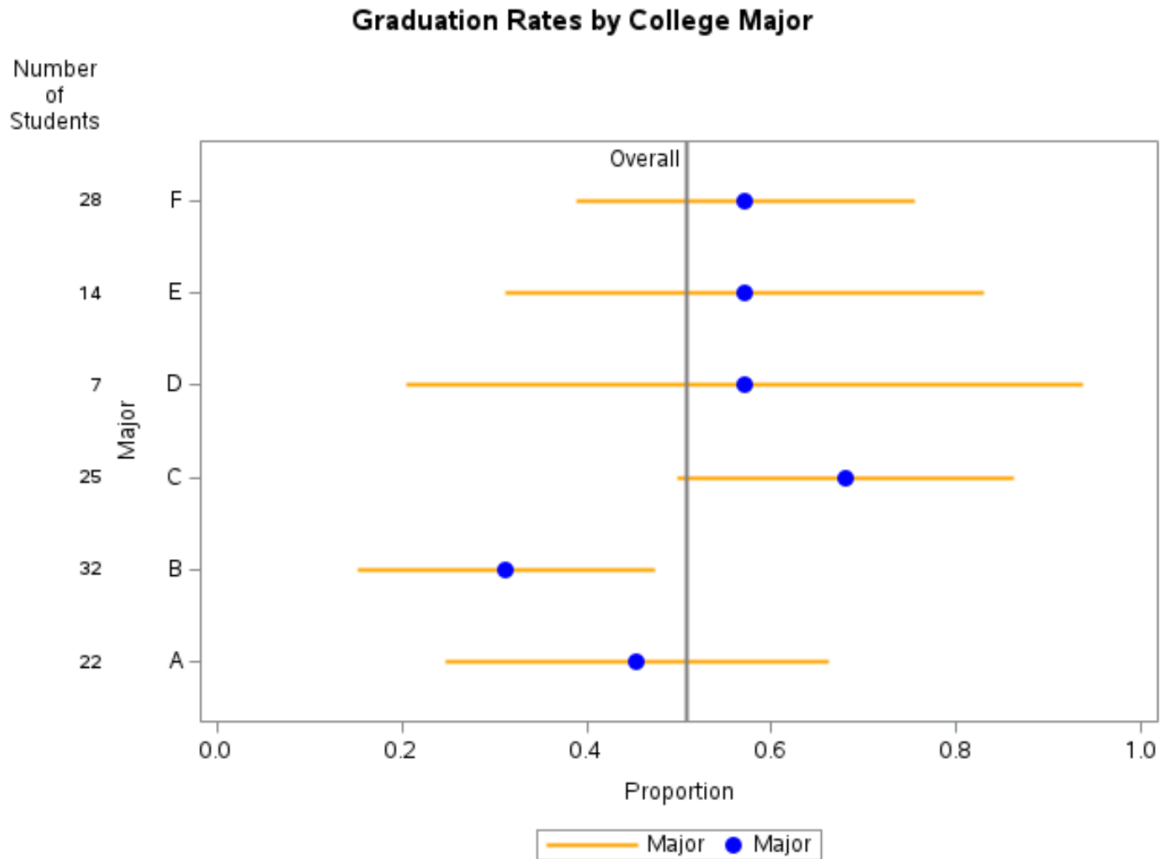
A typical plot of binomial proportions with confidence intervals shows:

- **Points:** estimated proportions
- **Error bars:** 95% confidence intervals
- **Vertical reference line:** overall proportion
- **Table entries:** number of students in each major

Such a graph clearly illustrates that majors with fewer students (for example, very small departments) have much wider confidence intervals than majors with larger enrollments.

8.4.5 Practical visualization advice

- If there are **10 or more categories**, consider using **alternating background color bands** to make it easier to associate confidence intervals with groups.
- Always label axes clearly (e.g., *Proportion* on the x-axis).
- Include the confidence level (e.g., “95% CI”) in the caption or title.



8.4.6 Summary

This section demonstrates how visualization complements numerical output:

- PROC FREQ provides estimated proportions and confidence intervals.
- Graphing proportions with confidence intervals makes comparisons intuitive.
- Including sample sizes helps readers understand why some intervals are wider than others.

Well-designed graphics are one of the most effective ways to communicate results from binomial proportion analyses.

9 SAS Macro Program in One Sample Variance Problem

Learning Objectives

1. Apply hypothesis testing and confidence interval construction for **one population variance**.
2. Use this example to motivate **writing reusable SAS macro programs**.

9.1 Motivation

In many applications, we may care more about the **population variance** σ^2 than the population mean μ . For instance, in a manufacturing process, a **large variance** in product measurements indicates unstable quality, even if the mean is acceptable. This is not the only instance, as there are many interesting scientific questions that involve the population variance both in academia and in industry.

9.1.1 Examples

A SCUBA instructor records the dive depths of students during checkout. Although everyone should be at the same depth, the instructor is interested in how much the depths vary.

The instructor believes the standard deviation is **3 feet**, while the assistant thinks it is **less than 3 feet**.

The hypotheses are:

$$H_0 : \sigma^2 = 3^2 \quad \text{vs} \quad H_1 : \sigma^2 < 3^2$$

A company produces metal pipes of a standard length. Twenty years ago, the pipe lengths were normally distributed with standard deviation **1.1 cm**.

The company now tests a random sample of **30 pipes** and wants to construct a **95% confidence interval** for σ^2 to determine whether the production quality has changed.

For the examples above, the quantity of the interest is the population variance. But we need something to estimate it. Let s^2 denote the sample variance. Then the pivotal quantity

$$\frac{(n-1)s^2}{\sigma^2},$$

which follows a chi-square distribution:

$$\frac{(n-1)s^2}{\sigma^2} \sim \chi_{n-1}^2,$$

where

- n : sample size
- s^2 : sample variance
- σ^2 : population variance

You may think that s as the random variable in this test. The distribution is called chi-square distribution which depends on a parameter named the *degrees of freedom* denoted by $df = n - 1$. A test of a single variance may be right-tailed, left-tailed or two-sided. This chi-square result is the foundation for both **confidence intervals** and **hypothesis tests** for σ^2 .

i Note

A $(1 - \alpha) \times 100\%$ confidence interval for the population variance is

$$\left(\frac{(n-1)s^2}{\chi_{1-\alpha/2, n-1}^2}, \frac{(n-1)s^2}{\chi_{\alpha/2, n-1}^2} \right),$$

where $\chi_{p, n-1}^2$ denotes the p -th quantile of the χ_{n-1}^2 distribution.
I won't expect you to know the expression here for now.

Problem

Unfortunately, if you check the built-in SAS procedures, you will find that **there is no basic procedure** that directly handles the **one-sample variance inference problem**.

The good news is that SAS provides a powerful **MACRO programming language**, which allows us to **write our own SAS procedures**.

9.2 SAS Macro Program

When you write a program that will be run **repeatedly**, you may want to seriously consider using **macros**, because:

- **MARCROS allow centralized changes:**
You can make a change in one location, and SAS will cascade that change throughout your program.
- **MACROS promote code reuse:**
You can write a section of code once and reuse it many times, either within the same program or across different programs.
- **MARCROS enable data-driven programming:**
SAS can decide what actions to take based on actual data values.

In general, macro code takes **longer to write and debug** than standard SAS code. Therefore, macros are usually **not recommended** for programs that will only be run a few times.

However, if you find yourself writing **similar code repeatedly**, macros can significantly improve efficiency and reduce errors.

Macros can help in several ways:

1. You can make a **single small change**, and SAS will propagate that change throughout your program.
2. You can **reuse code blocks**, avoiding duplication.
3. You can build **data-driven programs**, allowing SAS to adapt automatically to different datasets or inputs.

9.3 How Macros Work

When you submit a standard SAS program, SAS **compiles and executes it immediately**. When you write macro code, there is an **additional step**:

- SAS first sends MARCO statements to the **macro processor**.
- The macro processor **resolves the macro code**, generating standard SAS code.
- SAS then compiles and executes the generated code.

Because you are writing a program that **writes another program**, this process is sometimes referred to as **meta-programming**.

9.4 Designing Your Own Macros

To design effective SAS macros, we need to understand two fundamental concepts:

9.4.1 Macros vs. Macro Variables

As you construct macro programs, you will work with two basic building blocks:

- **MACROS** v.s.
- **MACRO variables**

To design your own SAS macros, we need to clearly distinguish between **macros** and **macro variables**. These are the two basic building blocks of SAS macro programming.

Conceptually, the workflow looks like this:

- You write **macro code**
- The **macro processor** resolves it
- The output is **ordinary SAS code**, which is then compiled and executed

Naming Conventions

You can tell macros and macro variables apart by their prefixes:

- **Macro variables** start with an ampersand: **&**
- **Macros** start with a percent sign: **%**

Examples:

- **&alpha**, **&n**
- **%myMacro**, **%DO**, **%IF**

Macro Variables

A **macro variable** is similar to a data variable, but with important differences:

- It does **not belong to a dataset**
- It has **only one value**
- That value is **always character**
- The value is **substituted directly into your program**

A macro variable's value can be:

- A variable name
- A number
- Text

- Any string you want substituted into your SAS code

Macro

A **macro** is a larger unit of code that can contain:

- DATA steps
- PROC steps
- Macro logic such as
%IF-%THEN-%ELSE, %DO-%END

Macros often—but not always—use macro variables.

9.5 Think Globally and Locally: Scope of Macro Variables

Macro variables come in two scopes:

- **Local macro variables**
- **Global macro variables**

9.5.1 Local Macro Variables

- Defined **inside a macro**
- Exist **only within that macro**
- Cannot be referenced outside the macro

9.5.2 Global Macro Variables

- Defined in **open code** (outside any macro)
- Can be used **anywhere in the program**

9.5.3 Common Mistakes to Avoid

- Using a local macro variable outside its macro
- Accidentally creating **both local and global macro variables with the same name**

Keeping scope in mind will save you a lot of debugging time.

Note

The macro processor **does not resolve macro references inside single quotes**.

- '&var' → macro not resolved
- "&var" → macro resolved

Rule of thumb:

Use **double quotes** when quoted strings contain macro variables.

Substituting Text with %LET

The %LET statement assigns a value to a macro variable.

```
%LET alpha = 0.05;

%MACRO macro_name(param1, param2, ..., paramk);
    /* macro code */
%MEND macro_name;
```

9.6 Writing SAS Macros with Car dataset

Before writing our own macros, let us first understand how **macro variables** work through a simple example.

SAS provides a built-in **global macro variable** called SYSDATE, which represents the system date.

Suppose we want to print the system date automatically in the title of a SAS report every time the report is generated. The advantage is that we do **not** need to manually update the date in the code. We use the built-in SAS dataset CARS from the SASHELP library.

```
PROC PRINT DATA=SASHELP.CARS;
    WHERE MAKE = 'Audi' AND TYPE = 'Sports';
    TITLE "Sales as of &SYSDAY &SYSDATE";
RUN;
```

Sales as of Saturday 14FEB26

Obs	Make	Model	Type	Origin	DriveTrain	MSRP	Invoice	EngineSize	Cylinders	Horsepower	MPG_City	MPG_Highway	Weight	Wheelbase	Length
21	Audi	RS 6 4dr	Sports	Europe	Front	\$84,600	\$76,417	4.2	8	450	15	22	4024	109	191
22	Audi	TT 1.8 convertible 2dr (coupe)	Sports	Europe	Front	\$35,940	\$32,512	1.8	4	180	20	28	3131	95	159
23	Audi	TT 1.8 Quattro 2dr (convertible)	Sports	Europe	All	\$37,390	\$33,891	1.8	4	225	20	28	2921	96	159
24	Audi	TT 3.2 coupe 2dr (convertible)	Sports	Europe	All	\$40,590	\$36,739	3.2	6	250	21	29	3351	96	159

So far, we have used macro variables to make our SAS programs more flexible.

Now we take one more step forward and define a **macro program**, which is a reusable block of SAS code. Macro variables are referenced in SAS statements using the ampersand (&) character.

Now suppose we want our program to be more flexible. Instead of hard-coding the car MAKE and TYPE, we define macro variables so that we can easily change them without modifying multiple lines of code.

```
%LET MAKE_NAME = Audi;
%LET TYPE_NAME = Sports;

PROC PRINT DATA=SASHELP.CARS;
  WHERE MAKE = "&MAKE_NAME" AND TYPE = "&TYPE_NAME";
  TITLE "Sales as of &SYSDAY &SYSDATE";
RUN;
```

Sales as of Saturday 14FEB26

Obs	Make	Model	Type	Origin	DriveTrain	MSRP	Invoice	EngineSize	Cylinders	Horsepower	MPG_City	MPG_Highway	Weight	Wheelbase	Length
21	Audi	RS 6 4dr	Sports	Europe	Front	\$84,600	\$76,417	4.2	8	450	15	22	4024	109	191
22	Audi	TT 1.8 convertible 2dr (coupe)	Sports	Europe	Front	\$35,940	\$32,512	1.8	4	180	20	28	3131	95	159
23	Audi	TT 1.8 Quattro 2dr (convertible)	Sports	Europe	All	\$37,390	\$33,891	1.8	4	225	20	28	2921	96	159
24	Audi	TT 3.2 coupe 2dr (convertible)	Sports	Europe	All	\$40,590	\$36,739	3.2	6	250	21	29	3351	96	159

Reusing the Same Program with Different Values

Now, suppose we want to view Audi vehicles of type Wagon. We only need to change one line of code.

```
/* Change only the macro variable values */
%LET MAKE_NAME = 'Audi';
%LET TYPE_NAME = 'Wagon';

/* The SAS code below remains unchanged */
PROC PRINT DATA = SASHELP.CARS;
  WHERE MAKE = &MAKE_NAME AND TYPE = &TYPE_NAME;
  TITLE "Sales as of &SYSDAY &SYSDATE";
RUN;
```

Sales as of Saturday 14FEB26

Obs	Make	Model	Type	Origin	DriveTrain	MSRP	Invoice	EngineSize	Cylinders	Horsepower	MPG_City	MPG_Highway	Weight	Wheelbase	Length
25	Audi	A6 3.0 Avant Quattro	Wagon	Europe	All	\$40,840	\$37,060	3.0	6	220	18	25	4035	109	192
26	Audi	S4 Avant Quattro	Wagon	Europe	All	\$49,090	\$44,446	4.2	8	340	15	21	3936	104	179

More on Macro program

The following program defines a macro called `show_result`.
This macro takes two arguments:

- `make_`: the car manufacturer
- `type_`: the vehicle type

```
%MACRO show_result(make_, type_);  
  
PROC PRINT DATA=SASHELP.CARS;  
  WHERE MAKE = "&make_" AND TYPE = "&type_";  
  TITLE "Sales as of &SYSDAY &SYSDATE";  
RUN;  
  
%MEND;
```

Calling a Macro Once a macro is defined, it can be called just like a function.

```
%show_result(BMW, SUV);
```

Sales as of Saturday 14FEB26															
Obs	Make	Model	Type	Origin	DriveTrain	MSRP	Invoice	EngineSize	Cylinders	Horsepower	MPG_City	MPG_Highway	Weight	Wheelbase	Length
27	BMW	X3 3.0i	SUV	Europe	All	\$37,000	\$33,873	3.0	6	225	16	23	4023	110	180
28	BMW	X5 4.4i	SUV	Europe	All	\$52,195	\$47,720	4.4	8	325	16	22	4824	111	184

So after seeing the first example, we have a more clear sense about why Use Macro Programs?

Macro programs allow you to:

- Avoid rewriting the same code repeatedly
- Centralize logic in one place
- Make programs easier to maintain and extend
- Write data-driven and parameterized SAS programs

This idea becomes especially powerful when we need to repeat the same analysis for many datasets or parameter values.

9.7 Revisit the One-Sample Variance Test

As discussed earlier, SAS does NOT provide a built-in procedure for conducting one-sample variance inference (confidence intervals or hypothesis tests). However, SAS allows us to overcome this limitation by writing our own macro procedures. Fortunately, a one-sample variance inference macro is commonly used and can be written in a general form. In this course, we will use a macro named `vartest.sas`, which is shared in the iCollege and below.

```

%macro vartest(version,
                data= _last_ ,
                var=      ,
                alpha=  0.05  ,
                var0=,
                sigma0=
                );

%if &version ne %then %put VARTEST macro Version 1.1;
%let opts = %sysfunc(getoption(notes))
            _last_=%sysfunc(getoption(_last_));
%if &data=_last_ %then %let data=&syslast;
options nonotes;

%if &sigma0=0 or &var0=0 %then %do;
    %put ERROR: The null hypothesis variance (VAR0=) or standard deviation;
    %put %str(      ) (SIGMA0=) must be greater than zero.;
    %goto exit;
%end;

proc summary data=&data;
    var &var;
    output out=_varn(keep=var n std) n=n var=var std=std;
run;

data _vartest;
    set _varn;
    df=n-1;
    %if &sigma0 ne %then %str(nullvar=&sigma0 * &sigma0);
    %if &var0 ne %then %str(nullvar=&var0);
    level=(1-&alpha)*100;
    call symput('level',trim(left(level)));
    %if &sigma0 ne or &var0 ne %then %do;
        chisq=((df)*(var))/nullvar;
        pvalue=1-probchi(chisq,df);
    %end;
    uclvar=df*var/cinv(&alpha/2,df); uclstd=sqrt(uclvar);
    lclvar=df*var/cinv(1-&alpha/2,df); lclstd=sqrt(lclvar);
    label var="Sample Variance" std="Sample Std Dev"
          chisq="Chi-Square" pvalue="Prob>Chi-Square"
          level="Confidence Level (%)"
          uclvar="UCL for Variance" lclvar="LCL for Variance"

```



```

        uclstd="UCL for Std Dev" lclstd="LCL for Std Dev";
    drop df %if &sigma0 ne or &var0 ne %then nullvar; ;
run;

data _ci;
    set _vartest;
    stat="Variance"; estimate=var; lcl=lclvar; ucl=uclvar; output;
    stat="Std Dev"; estimate=std; lcl=lclstd; ucl=uclstd; output;
    label stat="Statistic"
           estimate="Estimate"
           lcl="Lower Confidence Limit"
           ucl="Upper Confidence Limit";
run;

proc print data=_ci label noobs;
    var stat estimate lcl ucl;
    title "&level.% Confidence Intervals for Variance and Std Dev";
run;

%if &sigma0 ne or &var0 ne %then %do;
proc print data=_vartest label noobs;
    var chisq pvalue;
    format pvalue 6.4;
    title "Hypothesis test for";
    title2;
    %if &var0 ne %then %do;
        title3 "H0: Variance(%upcase(&var)) = &var0";
        title4 "Ha: Variance(%upcase(&var)) > &var0";
    %end;
    %if &sigma0 ne %then %do;
        title3 "H0: StdDev(%upcase(&var)) = &sigma0";
        title4 "Ha: StdDev(%upcase(&var)) > &sigma0";
    %end;
run;
%end;

%exit:
title;
options &opts;
%mend;

```

With this Macro, we can implement it to construct the hypothesis testing or confidence interval

for population variance σ^2 .

```
/* Create a sample data set */
DATA DAT;
    INPUT X @@;
    DATALINES;
6.2 1.9 4.4 4.9 3.5
4.6 4.2 1.1 1.3 4.8
4.1 3.7 2.5 3.7 4.2
1.4 3.2 2.6 1.5 3.9
;
RUN;

/* Display the data */
PROC PRINT DATA = DAT;
RUN;
```

This dataset includes **20 measurements** of the length of a product from a manufacturing process. Again, our goal is to conduct **statistical inference on the population variance** of the product length. Specifically, we want to test whether the population variance is **greater than 2.25**.

```
DATA DAT;
    INPUT X @@;
    DATALINES;
6.2 1.9 4.4 4.9 3.5
4.6 4.2 1.1 1.3 4.8
4.1 3.7 2.5 3.7 4.2
1.4 3.2 2.6 1.5 3.9
;
RUN;

PROC PRINT DATA = DAT;
RUN;
```

9.7.1 Using the VARTST Macro in SAS

Once the dataset has been created in SAS and the macro has been saved in a file with a known path, we can use the following code to conduct the **one-sample variance test** and construct the corresponding **confidence intervals**.

Use the %INCLUDE statement to load the macro into your SAS session:

```

/* When using SAS Windows */
%INCLUDE
"C:/Users/cyeh/Desktop/Teaching/GSU/STAT8678-SAS/code/L10/vartest.sas";

/* in SAS studio */
%INCLUDE
"/home/u64378616/chikuang/Part2/L10_Marco/vartest.sas";

%VARTST(
    DATA    = DAT,
    VAR      = X,
    ALPHA    = 0.05,
    VARO     = 2.25
);

```

95% Confidence Intervals for Variance and Std Dev

Statistic	Estimate	Lower Confidence Limit	Upper Confidence Limit
Variance	1.98661	1.14894	4.23796
Std Dev	1.40947	1.07189	2.05863

Hypothesis test for

H0: Variance(X) = 2.25

Ha: Variance(X) > 2.25

Chi-Square	Prob>Chi-Square
16.7758	0.6051

Question for you:

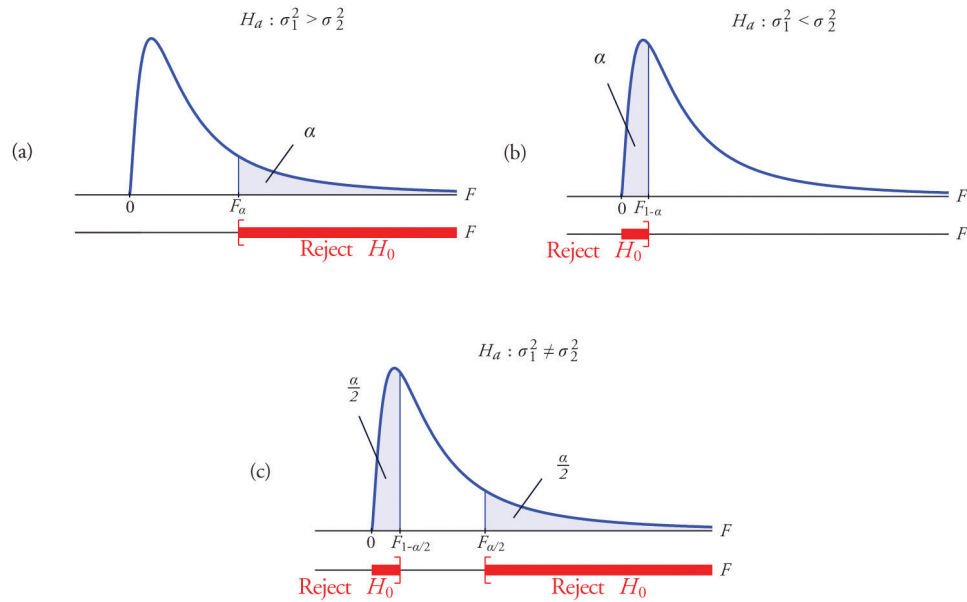
How would you modify the macro call if you want to test whether the population standard deviation is greater than 1.5 instead of testing the population variance?

i Answer with Explanation

To calculate the p-value for $H_1 : \sigma^2 < \sigma_0^2$ and $H_1 : \sigma^2 \neq \sigma_0^2$ from the p-value for $H_1 : \sigma^2 > \sigma_0^2$. We can do the following:

Let p be the p-value for the right-tailed test $H_1 : \sigma^2 > \sigma_0^2$.

- For the left-tailed test $H_1 : \sigma^2 < \sigma_0^2$, the p-value is $1 - p$.
- For the two-sided test $H_1 : \sigma^2 \neq \sigma_0^2$, the p-value is $\min\{p, 1 - p\} \times 2$.



10 χ^2 Goodness of Fit Test

Learning Objectives

1. Explain what is a diagnostic test.
2. Explain what is a one-way contingency table.
3. Apply the diagnostic test to perform inference for analyzing a one-way contingency table.

10.1 Motivation: What Is a Goodness-of-Fit Test?

A **goodness-of-fit test**, in general, refers to measuring how well the observed data correspond to a fitted (assumed) model.

We will use this concept throughout the course as a way of checking model fit. Similar to linear regression, in essence, a goodness-of-fit test compares:

- **Observed values**, and
- **Expected (fitted or predicted) values** under a specified model.

Hypotheses for a Goodness-of-Fit Test

A goodness-of-fit test examines the following hypotheses:

$$H_0 : \text{the model } M_0 \text{ fits}$$

versus

$$H_1 : \text{the model } M_0 \text{ does not fit.}$$

10.2 Example: Categorical Data (Dice Example)

Consider the simplest example of a goodness-of-fit test with **categorical data**.

Suppose we want to test whether a fair six-sided die has **equal probability** for each face. We compare the observed frequencies to those expected under the assumed model:

$$X \sim \text{Multinomial} \left(n = 30, \pi_0 = \left(\frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6} \right) \right).$$

In this setting, the goodness-of-fit test asks whether the probability in each cell is equal to the specified value π_0 .

10.3 Hypotheses in Probability Vector Form

The hypotheses can be written as:

$$H_0 : \pi = \pi_0$$

versus

$$H_1 : \pi \neq \pi_0.$$

10.4 One-Way Contingency Table

In such problems, the data can be summarized using a **one-way contingency table**:

Categories	1	2	3	4	5	6
Number of Observations	O_1	O_2	O_3	O_4	O_5	O_6

Here,

- $O_1, O_2, \dots, O_6$ are **observed counts** from your dataset.
- The expected counts are computed from the assumed probability model.

These observed and expected counts form the basis of the χ^2 goodness-of-fit test.

10.5 One-Way Contingency Table and Model Fit

Categories	1	2	3	4	5	6
Number of Observations	O_1	O_2	O_3	O_4	O_5	O_6
Expected Observations	$n\pi_1 = 5$	5	5	5	5	5

In the setting of one-way contingency tables, we measure how well an observed variable X corresponds to a model specified by a vector of cell probabilities.

In other words, under the null hypothesis, we assume the data come from a particular distribution, and we test whether this assumed model fits the data well when compared with the **saturated model** (the model that fits the data perfectly).

The rationale behind model fitting is the assumption that a complex data-generating mechanism can be reasonably approximated by a simpler model. The goodness-of-fit test is applied to assess whether this assumption is supported by the data.

10.6 Test Statistics

There are two commonly used test statistics for the χ^2 goodness-of-fit test.

Pearson Goodness-of-Fit Test Statistic

The Pearson goodness-of-fit statistic is defined as

$$X^2 = \sum_{j=1}^k \frac{(X_j - n\pi_{0j})^2}{n\pi_{0j}}.$$

An easy way to remember this formula is

$$X^2 = \sum_{j=1}^k \frac{(O_j - E_j)^2}{E_j},$$

where

- $O_j = X_j$ is the observed count in cell j , and
- $E_j = \mathbb{E}(X_j) = n\pi_{0j}$ is the expected count in cell j under the null hypothesis.

10.6.1 Deviance Test Statistic

The deviance statistic is defined as

$$G^2 = 2 \sum_{j=1}^k X_j \log\left(\frac{X_j}{n\pi_{0j}}\right) = 2 \sum_{j=1}^k O_j \log\left(\frac{O_j}{E_j}\right).$$

In some texts, G^2 is also referred to as the **likelihood ratio test (LRT) statistic**, which compares the log-likelihoods of two models:

- L_0 : the reduced model under H_0 , and
- L_1 : the full (saturated) model under H_A .

$$G^2 = -2 \log\left(\frac{\ell_0}{\ell_1}\right) = -2(L_0 - L_1)$$

Note that X^2 and G^2 are both functions of the observed data X and a vector of cell probabilities π_0 . For this reason, we will sometimes write them as $X^2(x, \pi_0)$ and $G^2(x, \pi_0)$, respectively. When there is no ambiguity, we will simply use X^2 and G^2 .

We will encounter these statistics repeatedly throughout the course, particularly in the analysis of two-way and k -way contingency tables, and when assessing the fit of log-linear and logistic regression models.

10.7 How Do They Work?

Both X^2 and G^2 measure how closely the assumed model—here, a multinomial model $\text{Mult}(n, \pi_0)$ —fits the observed data.

When the null hypothesis H_0 is true, both statistics have an approximate chi-square distribution with $k - 1$ degrees of freedom. This allows us to use the chi-square distribution to obtain critical values and p -values for hypothesis testing.

- If the sample proportions $\hat{\pi}_j$ (i.e., the saturated model) are exactly equal to the model probabilities π_{0j} for all cells $j = 1, 2, \dots, k$, then $O_j = E_j$ for all j , and both X^2 and G^2 are equal to zero. In this case, the model fits the data perfectly.
- If the sample proportions $\hat{\pi}_j$ deviate from the model probabilities π_{0j} , then both X^2 and G^2 are positive. Large values of X^2 and G^2 indicate that the data do not agree well with the assumed model M_0 .

10.8 Dice Rolls Example

Suppose that we roll a die 30 times and observe the following table showing the number of times each face appears.

Categories	1	2	3	4	5	6
Number of Observations	$O_1 = 3$	$O_2 = 7$	$O_3 = 5$	$O_4 = 10$	$O_5 = 2$	$O_6 = 3$
Expected Observations						

We want to test the null hypothesis that the die is fair.

Under this hypothesis,

$$X \sim \text{Mult}(n = 30, \pi_0),$$

where

$$\pi_{0j} = \frac{1}{6}, \quad j = 1, \dots, 6.$$

This is our assumed model. Under H_0 , the expected counts are

$$E_j = \frac{30}{6} = 5 \quad \text{for each cell.}$$

We now have all the quantities needed to compute the goodness-of-fit statistics.

We now compute the two goodness-of-fit statistics explicitly.

10.8.1 Pearson chi-square statistic

$$\begin{aligned}
 X^2 &= \frac{(3-5)^2}{5} + \frac{(7-5)^2}{5} + \frac{(5-5)^2}{5} + \frac{(10-5)^2}{5} + \frac{(2-5)^2}{5} + \frac{(3-5)^2}{5} \\
 &= 9.2
 \end{aligned}$$

10.8.2 Deviance (likelihood-ratio) statistic

$$\begin{aligned} G^2 &= 2 \left(3 \log \frac{3}{5} + 7 \log \frac{7}{5} + 5 \log \frac{5}{5} \right. \\ &\quad \left. + 10 \log \frac{10}{5} + 2 \log \frac{2}{5} + 3 \log \frac{3}{5} \right) \\ &= 8.8 \end{aligned}$$

Note that although X^2 and G^2 have the same approximate chi-square distribution, their realized numerical values need not be identical.

The corresponding p -values are

$$P(\chi_5^2 \geq 9.2) = 0.10, \quad P(\chi_5^2 \geq 8.8) = 0.12.$$

Given a significance level of $\alpha = 0.05$, we **fail to reject** the null hypothesis.

Importantly, failing to reject H_0 does **not** mean that the null hypothesis is true. Rather, it means that we do not have sufficient evidence to conclude that it is false.

In this example, the fair-die model does not fit the data exactly, but the lack of fit is not large enough to conclude that the die is unfair at the 5% significance level.

The following SAS code implements the chi-square goodness-of-fit test for the dice example.

SAS code: Chi-square goodness-of-fit test

```
DATA DIE;
    INPUT FACE $ COUNT;
    DATALINES;
1 3
2 7
3 5
4 10
5 2
6 3
;
RUN;

PROC FREQ DATA=DIE;
    WEIGHT COUNT;
    TABLES FACE / CHISQ;
/*
```

```

TABLES FACE / NOCUM ALL
      CHISQ TESTP=(16.67 16.67 16.67 16.67 16.67 16.67);
      */
RUN;

```

The FREQ Procedure

FACE	Frequency	Percent	Cumulative Frequency	Cumulative Percent
1	3	10.00	3	10.00
2	7	23.33	10	33.33
3	5	16.67	15	50.00
4	10	33.33	25	83.33
5	2	6.67	27	90.00
6	3	10.00	30	100.00

Chi-Square Test for Equal Proportions	
Chi-Square	9.2000
DF	5
Pr > ChiSq	0.1013

The PROC FREQ step computes the Pearson chi-square statistic for testing whether the die

is fair. The commented TESTP= option shows how expected probabilities can be specified explicitly.

Computing residuals and goodness-of-fit statistics manually

We now compute Pearson residuals, deviance residuals, and the test statistics X^2 and G^2 directly from the data.

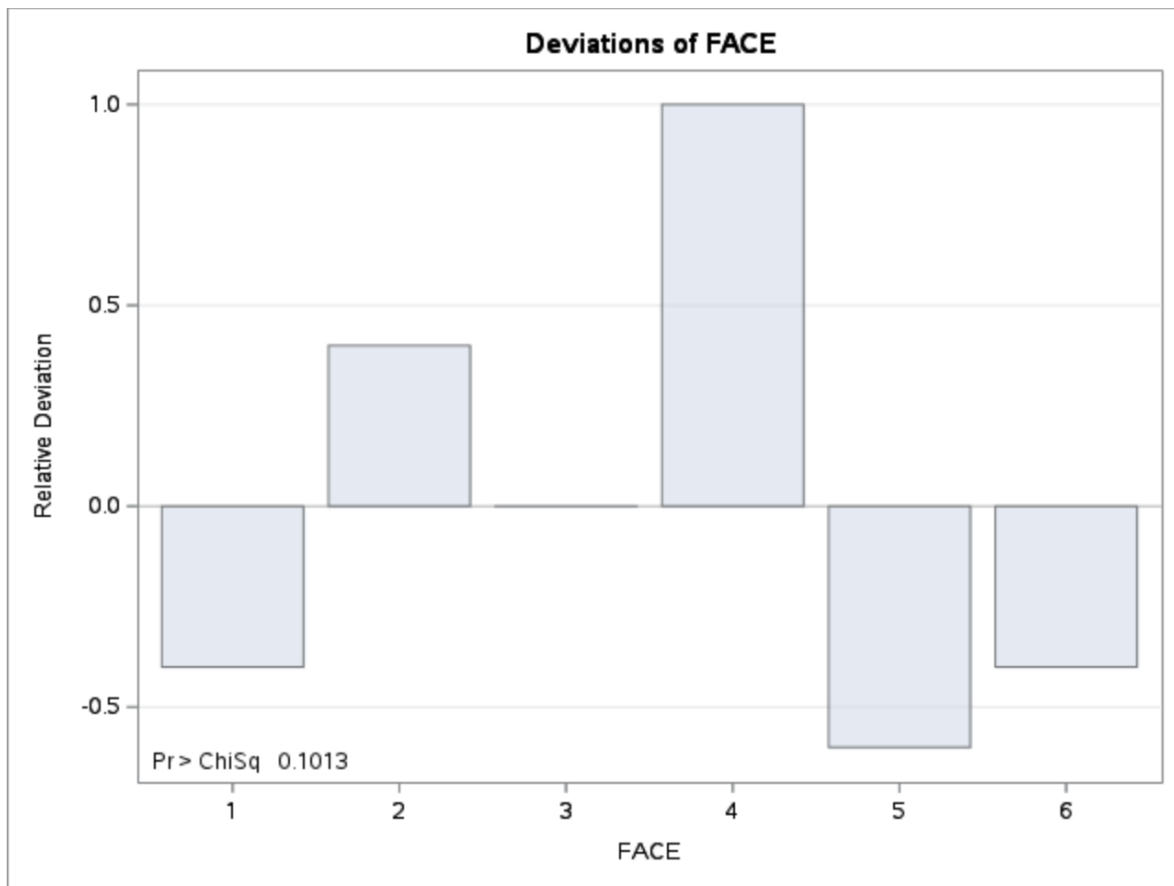
```
DATA CAL;
  SET DIE;

  PI = 1/6;
  ECOUNT = 30 * PI;

  /* Pearson residual */
  RES = (COUNT - ECOUNT) / SQRT(ECOUNT);

  /* Deviance residual */
  DEVRES = SQRT(ABS(2 * COUNT * LOG(COUNT / ECOUNT)))
    * SIGN(COUNT - ECOUNT);
RUN;

PROC PRINT DATA=CAL;
RUN;
```



Sample Size = 30

Obs	FACE	COUNT	PI	ECOUNT	RES	DEVRES
1	1	3	0.16667	5	-0.89443	-1.75070
2	2	7	0.16667	5	0.89443	2.17039
3	3	5	0.16667	5	0.00000	0.00000
4	4	10	0.16667	5	2.23607	3.72330
5	5	2	0.16667	5	-1.34164	-1.91446
6	6	3	0.16667	5	-0.89443	-1.75070

Computing test statistics and p -values

```

PROC SQL;
  SELECT
    SUM((COUNT - ECOUNT)**2 / ECOUNT) AS X2,
    1 - PROBCHI(CALCULATED X2, 5)      AS PVALUE_X2,
    2 * SUM(COUNT * LOG(COUNT / ECOUNT)) AS G2,
    1 - PROBCHI(CALCULATED G2, 5)      AS PVALUE_G2
  FROM CAL;
QUIT;

```

X2	PVALUE_X2	G2	PVALUE_G2
9.2	0.101348	8.778485	0.118233

Note that PROC FREQ automatically reports the Pearson chi-square statistic, but does not directly report the deviance statistic. The deviance statistic G^2 must be computed manually as shown above.

10.9 Tomato Phenotypes Example

Tall cut-leaf tomatoes were crossed with dwarf potato-leaf tomatoes, and $n = 1611$ offspring were classified by their phenotypes as summarized in the table below.

Categories	tall cut-leaf	tall potato-leaf	dwarf cut-leaf	dwarf potato-leaf
Number of Observations	$O_1 = 926$	$O_2 = 288$	$O_3 = 293$	$O_4 = 104$
Expected Observation				

Genetic theory predicts that the four phenotypes should occur with relative frequencies

$$9 : 3 : 3 : 1,$$

which implies that the phenotypes are **not equally likely**. In particular, the dwarf potato-leaf phenotype is expected to be the least frequent.

We would like to test whether the observed data are consistent with this genetic theory.

10.9.1 Null hypothesis

Under the null hypothesis, the cell probabilities are

$$\pi_1 = \frac{9}{16}, \quad \pi_2 = \pi_3 = \frac{3}{16}, \quad \pi_4 = \frac{1}{16}.$$

10.9.2 Expected frequencies

The expected counts under H_0 are

$$\begin{aligned} E_1 &= 1611 \times \frac{9}{16} = 906.2, \\ E_2 &= E_3 = 1611 \times \frac{3}{16} = 302.1, \\ E_4 &= 1611 \times \frac{1}{16} = 100.7. \end{aligned}$$

10.9.3 Goodness-of-fit results

Using these expected counts, we compute the goodness-of-fit statistics and obtain

$$X^2 = 1.47, \quad G^2 = 1.48.$$

The two statistics are nearly identical. Under the chi-square distribution with $k - 1 = 3$ degrees of freedom, the corresponding p -values are approximately

$$p \approx 0.69.$$

Since the p -value is much larger than the significance level $\alpha = 0.05$, we **fail to reject** the null hypothesis. The observed data are therefore **consistent with the genetic theory** predicting a 9 : 3 : 3 : 1 ratio.

10.9.3.1 SAS Code: Goodness-of-Fit Test

Step 1: Enter the observed counts

```

DATA LEAF;
  INPUT TYPE $ COUNT;
  DATALINES;
tallc    926
tallp    288
dwarfc   293
dwarfp   104
;
RUN;

```

The FREQ Procedure

TYPE	Frequency	Percent	Test Percent	Cumulative Frequency	Cumulative Percent
tallc	926	57.48	56.25	926	57.48
tallp	288	17.88	18.75	1214	75.36
dwarfc	293	18.19	18.75	1507	93.54
dwarfp	104	6.46	6.25	1611	100.00

Step 2: Pearson chi-square goodness-of-fit test

```

PROC FREQ DATA=LEAF ORDER=DATA;
  WEIGHT COUNT;
  TABLES TYPE / ALL CHISQ
    TESTP=(56.25 18.75 18.75 6.25);
RUN;

```

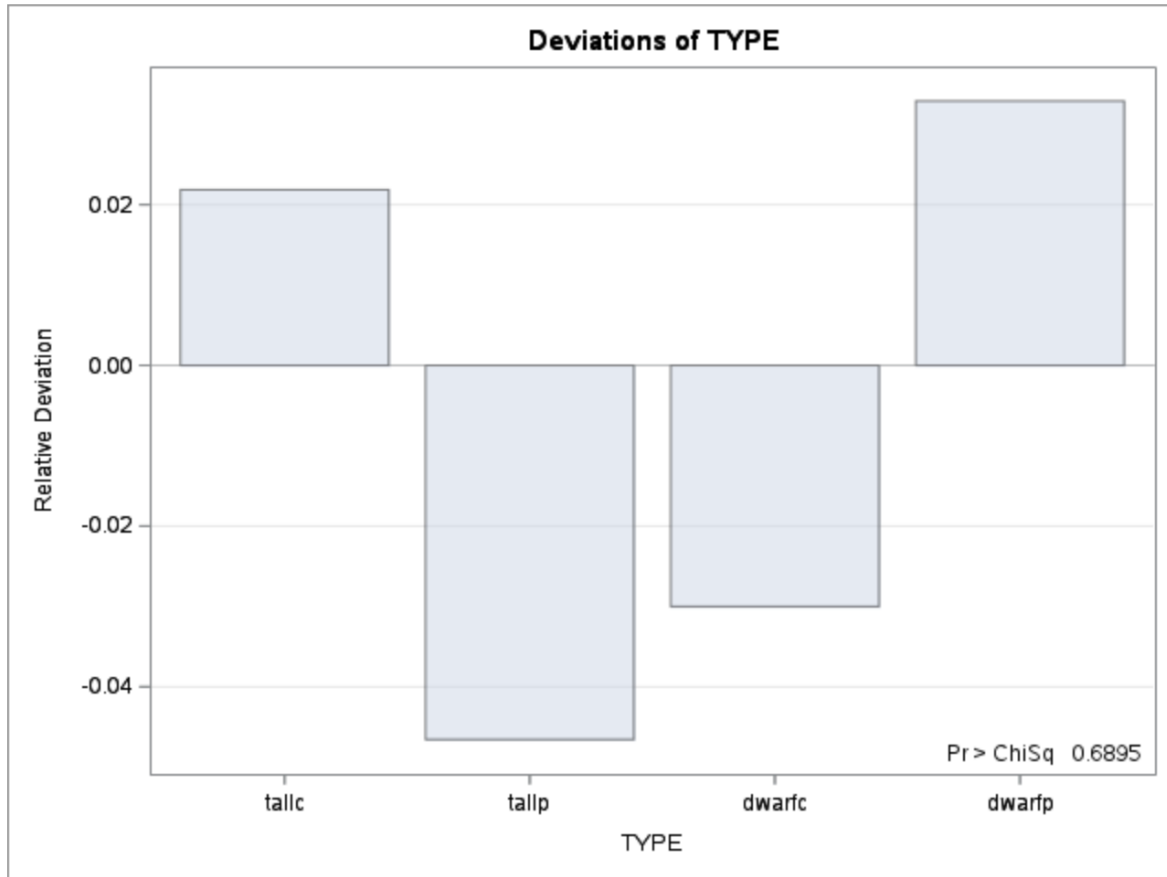
The proportions (56.25, 18.75, 18.75, 6.25) correspond to the theoretical probabilities (9,3,3,1)/16.

Manual Computation of Residuals, X^2 , and G^2

Chi-Square Test for Specified Proportions	
Chi-Square	1.4687
DF	3
Pr > ChiSq	0.6895

Step 3: Define theoretical probabilities

```
DATA PI;
    INPUT PI;
    PI = PI / 16;
CARDS;
9
3
3
1
;
RUN;
```



Sample Size = 1611

Step 4: Compute expected counts and residuals

```
DATA CAL;
  MERGE LEAF PI;
  ECOUNT = 1611 * PI;

  /* Pearson residual */
  RES = (COUNT - ECOUNT) / SQRT(ECOUNT);

  /* Deviance residual */
  DEVRES = SQRT(ABS(2 * COUNT * LOG(COUNT / ECOUNT)))
    * SIGN(COUNT - ECOUNT);
RUN;

PROC PRINT DATA=CAL;
RUN;
```

Obs	TYPE	COUNT	PI	ECOUNT	RES	DEVRES
1	tallc	926	0.5625	906.188	0.65816	6.32891
2	tallp	288	0.1875	302.063	-0.80912	-5.24022
3	dwarfc	293	0.1875	302.063	-0.52143	-4.22497
4	dwarfp	104	0.0625	100.688	0.33012	2.59476

Step 5: Compute X^2 and G^2 explicitly

```
PROC SQL;
  SELECT
    SUM((COUNT - ECOUNT)**2 / ECOUNT) AS X2,
    1 - PROBCHI(CALCULATED X2, 3)      AS PVAL1,
    2 * SUM(COUNT * LOG(COUNT / ECOUNT)) AS G2,
    1 - PROBCHI(CALCULATED G2, 3)      AS PVAL2
  FROM CAL;
QUIT;
```

X2	PVAL1	G2	PVAL2
1.468722	0.689508	1.477587	0.687453

Visualization: Observed vs Expected Counts

```
GOPTIONS RESET=ALL;
SYMBOL1 V=CIRCLE C=BLUE I=NONE;
SYMBOL2 V=CIRCLE C=RED I=NONE;

LEGEND1 LABEL=NONE
  VALUE=('Observed Count' 'Expected Count')
  ACROSS=1
  POSITION=(TOP RIGHT INSIDE)
  MODE=PROTECT;
```

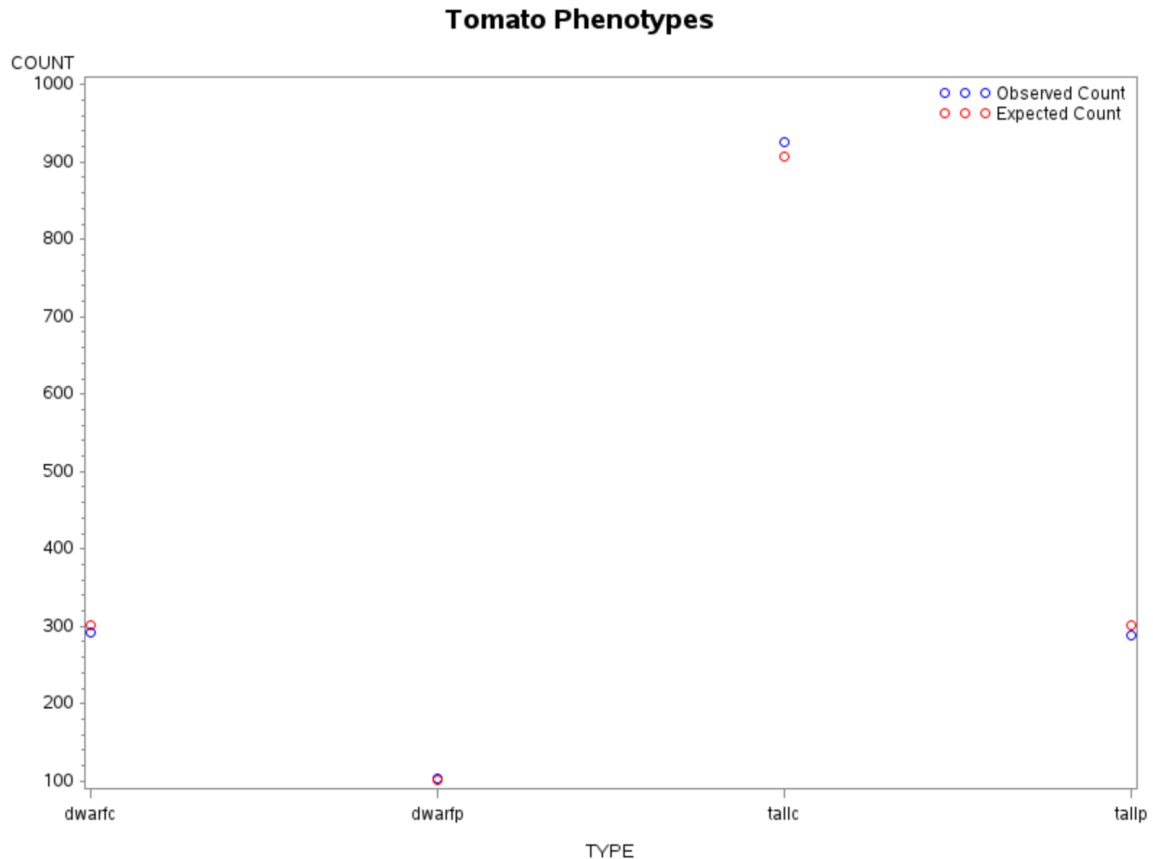
```

TITLE "Tomato Phenotypes";

PROC GPLOT DATA=CAL;
    PLOT COUNT*TYPE ECOUNT*TYPE / OVERLAY LEGEND=LEGEND1;
RUN;
QUIT;

TITLE;

```



Interpretation

- Small values of X^2 and G^2 indicate good agreement with the model
- Large p -values confirm no strong evidence against the 9 : 3 : 3 : 1 ratio
- Both Pearson and deviance statistics lead to the same conclusion

This example illustrates how goodness-of-fit testing connects theory, computation, and visualization in categorical data analysis.

11 Power Analysis with application in one sample t -test

Learning Objectives

1. Understand what power analysis is.
2. Explain why and when we need power analysis.

11.1 1 Introduction

A **power analysis** is a calculation that helps you determine the **minimum sample size** required for a study to reliably detect an effect of interest.

A power analysis is built around **four main components**.

If you know (or can reasonably estimate) **any three** of them, you can solve for the **fourth**.

11.1.1 Key Components of Power Analysis

- **Statistical power**

The probability that a statistical test will correctly detect an effect of a certain size *when the effect truly exists*.

Power is commonly set at **80% (0.8)** or higher.

- **Sample size**

The minimum number of observations needed to detect an effect of a given size with a specified power level.

- **Significance level (α)**

The maximum probability of rejecting a true null hypothesis (Type I error).

Typically set at $\alpha = 0.05$.

- **Expected effect size**

A standardized measure that quantifies the magnitude of the expected effect.

This is often based on:

- prior studies,

- pilot data, or
- subject-matter considerations.

11.2 Motivation Examples

A company that manufactures light bulbs claims that a particular type of light bulb lasts **850 hours on average**, with a **standard deviation of 50 hours**.

A consumer protection group believes that the manufacturer has **overestimated the average lifespan by about 40 hours**.

Question:

How many light bulbs does the consumer protection group need to test in order to demonstrate this discrepancy with reasonable confidence?

This is a classic **sample size determination problem**, where:

- the effect size is the difference between the claimed mean and the suspected true mean,
- the variability is known or estimated,
- and power and significance level must be specified.

It has been estimated that the average height of **American white male adults** is **70 inches**.

Suppose it is postulated that there is a **positive correlation between height and intelligence**.

If this is true, then the average height of **white male graduate students** on campus should be **greater than 70 inches**.

You plan to test this hypothesis by randomly sampling a group of white male graduate students.

Key question:

How small can the sample be (or how few individuals do you need to measure) and still have enough power to detect a meaningful difference?

This motivates power analysis for a **one-sided one-sample t -test**, where:

- the null hypothesis specifies a reference mean,
- the alternative hypothesis specifies a directional effect,
- and the goal is to balance feasibility (small sample size) with statistical reliability (adequate power).

In the remainder of this lecture, we will formalize these ideas and use the one-sample t -test to illustrate how power, sample size, effect size, and significance level are mathematically connected.

11.3 Before We Start the Analysis

For the power analysis in this lecture, we focus on the first example, which concerns testing the *average lifespan of a light bulb*. Our **first goal** is to determine the **number of light bulbs that must be tested**. That is, we want to find the **sample size** for a given significance level and power. Next, we will reverse the process and determine the **power**, given a sample size and significance level.

We know that the manufacturer claims the average lifespan of the light bulb is **850 hours**, with a **standard deviation of 50 hours**, while the consumer protection group believes that the manufacturer has overestimated the lifespan by about **40 hours**.

In terms of hypotheses:

- Null hypothesis: $H_0 : \mu = 850$
- Alternative hypothesis: $H_A : \mu = 810$

The **significance level** is the probability of a **Type I error**, that is, rejecting H_0 when it is actually true. We will set $\alpha = 0.05$.

The **power of the test** against H_A is the probability that the test **rejects H_0 when H_A is true**. We will set the desired power level to $\text{Power} = 0.90$.

Before proceeding with the power analysis, let us briefly discuss the role of the **standard deviation**. Intuitively, the number of light bulbs required for testing depends on the **variability** of their lifespans:

- If all light bulbs had exactly the same lifespan, then testing a single bulb would suffice.
- In reality, lifespans vary substantially. For example, some bulbs may last 1000 hours, while others may last 500 hours.

Therefore, the **standard deviation of the lifespan distribution** plays a critical role in determining the required sample size.

11.4 Power Analysis in SAS

In SAS, it is straightforward to perform a power analysis for **comparing means** using PROC POWER.

To compute the required sample size:

1. Specify the mean under the null hypothesis.
2. Specify the mean under the alternative hypothesis.
3. Specify the population standard deviation.
4. Set the significance level α (default is 0.05).

5. Specify the desired power level. Here we set $\text{Power} = 0.90$.
6. Indicate that the test is a **one-sample *t*-test**.

The following SAS code performs a power analysis for **Example 1**:

```
PROC POWER;  
  ONESAMPLEMEANS TEST=t  
    NULLMEAN = 850  
    MEAN      = 810  
    STDDEV    = 50  
    POWER     = 0.9  
    NTOTAL    = .;  
RUN;
```

This code tells SAS to solve for the total sample size (ntotal) needed to achieve 90% power for a one-sample *t*-test at significance level ($\alpha = 0.05$).

The POWER Procedure

One-Sample t Test for Mean

Fixed Scenario Elements	
Distribution	Normal
Method	Exact
Null Mean	850
Mean	810
Standard Deviation	50
Nominal Power	0.9
Number of Sides	2
Alpha	0.05

Computed N Total	
Actual Power	N Total
0.909	19

The result tells us that we need a **sample size of at least 19 light bulbs** in order to reject H_0 under the alternative hypothesis H_A with a **power of 0.9**.

Next, suppose we have a sample of size $n = 10$. How much power do we have if we keep all other quantities the same? We can use the same SAS program to calculate the power.

```
PROC POWER;  
  ONESAMPLEMEANS TEST=t  
    NULLMEAN = 850  
    MEAN      = 810  
    STDDEV    = 50  
    POWER     = .  
    NTOTAL    = 10;  
RUN;
```

The POWER Procedure

One-Sample t Test for Mean

Fixed Scenario Elements	
Distribution	Normal
Method	Exact
Null Mean	850
Mean	810
Standard Deviation	50
Total Sample Size	10
Number of Sides	2
Alpha	0.05

Computed Power
Power
0.616

You can see that the power is about 0.616 for a sample size of 10.

This means that if we only test 10 light bulbs, we have about a 61.6% chance of correctly rejecting the null hypothesis when the true mean is 810 hours. This is below the commonly desired power level of 0.8, indicating that a sample size of 10 may not be sufficient to reliably detect the effect size of interest.

What if we have sample size of 15 or 20? We can use a list of sample sizes as input to PROC POWER.

```
PROC POWER;  
  ONESAMPLEMEANS TEST=t  
    NULLMEAN = 850  
    MEAN      = 810  
    STDDEV    = 50  
    POWER     = .  
    NTOTAL    = 10 to 45 by 5;  
RUN;
```

The POWER Procedure

One-Sample t Test for Mean

Fixed Scenario Elements	
Distribution	Normal
Method	Exact
Null Mean	850
Mean	810
Standard Deviation	50
Number of Sides	2
Alpha	0.05

Computed Power		
Index	N Total	Power
1	10	0.616
2	15	0.821
3	20	0.924
4	25	0.970
5	30	0.988
6	35	0.996
7	40	0.999
8	45	>.999

We can also expect that if we actually know the standard deviation is smaller, we would need fewer light bulbs to achieve the same power. We can vary the standard deviation in PROC POWER to see how it affects the required sample size.

```
PROC POWER;  
  ONESAMPLEMEANS TEST=t  
    NULLMEAN = 850  
    MEAN      = 810  
    STDDEV    = 30 to 100 by 10  
    POWER     = 0.8  
    NTOTAL    = . ;  
RUN;
```

The POWER Procedure

One-Sample t Test for Mean

Fixed Scenario Elements	
Distribution	Normal
Method	Exact
Null Mean	850
Mean	810
Nominal Power	0.8
Number of Sides	2
Alpha	0.05

Computed N Total			
Index	Std Dev	Actual Power	N Total
1	30	0.834	7
2	40	0.803	10
3	50	0.821	15
4	60	0.807	20
5	70	0.815	27
6	80	0.808	34
7	90	0.803	42
8	100	0.808	52

Here, varying `STDDEV` to show how reduced variability lowers required sample size, while keeping the hypothesis, effect size, and power fixed.

11.5 Discussion

Normality assumption

One technical assumption underlying the power analysis: the **normality assumption**. If the variable of interest is not normally distributed, a small sample size will usually *not* achieve the power indicated by the theoretical results, because those results are derived under normality. In such cases, it may not even be appropriate to conduct a one-sample *t*-test with a very small sample size when the normality assumption is questionable.

Relative difference

There is another important technical point. What truly matters for power analysis is **not the individual values**, but the **difference between the two means**, relative to the variability of the data. In fact, what really determines the power is the ratio

$$\frac{\text{difference in means}}{\text{standard deviation}},$$

which is referred to as the **effect size**.

For example, we would obtain the same power if we subtracted 800 from both means, changing 850 to 50 and 810 to 10. The absolute scale of the data does not affect the power, only the standardized difference does.

```
PROC POWER;  
  ONESAMPLEMEANS TEST=t  
    NULLMEAN = 50  
    MEAN      = 10  
    STDDEV    = 50  
    POWER     = 0.9  
    NTOTAL    = . ;  
RUN;
```

If we standardize our variable, we can calculate the means in terms of change in standard deviation.


```

PROC POWER;
  ONESAMPLEMEANS TEST=t
    NULLMEAN = 1
    MEAN      = 0.2
    STDDEV    = 1
    POWER     = 0.9
    NTOTAL    = . ;
RUN;

```

i Key Takeaway

It is usually not an easy task to determine the “**true**” **effect size**. In practice, we make our best guess based on existing literature or evidence from a pilot study. A good estimate of the effect size is **crucial** for a successful power analysis.

11.6 Math behind the power analysis

- We saw earlier that in tests of significance there are two types of errors. Each error has a given probability of occurring.

- The **significance level** (Type I error) is

$$\alpha = P(\text{reject } H_0 \mid H_0 \text{ is true}).$$

- The probability of a **Type II error** is

$$\beta = P(\text{fail to reject } H_0 \mid H_0 \text{ is false}).$$

- The **power of a test** is related to the probability of a Type II error. It is defined as

$$\begin{aligned}
 1 - \beta &= 1 - P(\text{fail to reject } H_0 \mid H_0 \text{ is false}) \\
 &= P(\text{reject } H_0 \mid H_0 \text{ is false}).
 \end{aligned}$$

- Therefore, to define power, we must be specific about what “**when H_0 is false**” means.

11.7 A Visualization Example

Bottles of a popular cola drink are supposed to contain 300 ml of cola.

There is some variation from bottle to bottle because the filling machine is not perfectly precise.

Suppose the distribution of the contents is normal with standard deviation

$$\sigma = 3 \text{ ml.}$$

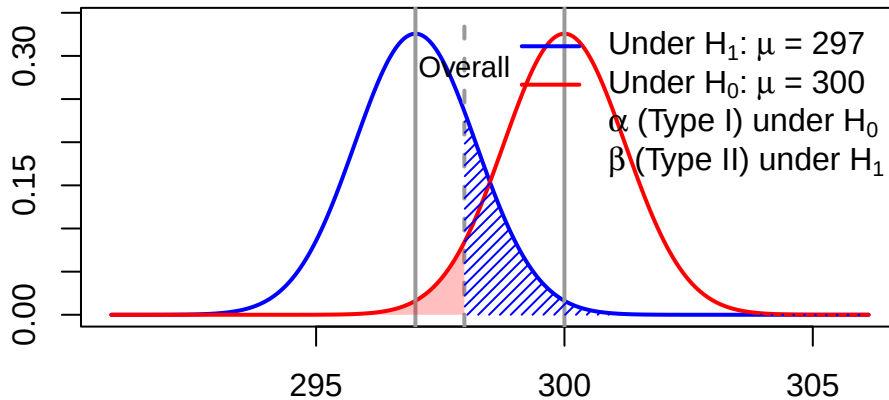
We want to carry out the following hypothesis test at significance level $\alpha = 0.05$, based on a sample of six bottles of cola:

$$H_0 : \mu = 300 \quad \text{versus} \quad H_1 : \mu < 300.$$

If the power is calculated under the alternative hypothesis with

$$\mu = 297,$$

then we can visualize α and β as follows.



11.7.1 Interpretation of the Power Visualization

The figure above illustrates the concepts of **Type I error** (α), **Type II error** (β), and **power** ($1 - \beta$) for a **left-tailed one-sample t -test**.

- The **red curve** represents the sampling distribution of the test statistic under the **null hypothesis**

$$H_0 : \mu = 300.$$

- The **blue curve** represents the sampling distribution under the **alternative hypothesis**

$$H_1 : \mu = 297.$$

- The **vertical cutoff line** corresponds to the critical value determined by the significance level $\alpha = 0.05$.

Type I Error (α)

- The **red shaded region** to the left of the cutoff is

$$\alpha = P(\text{reject } H_0 \mid H_0 \text{ is true}).$$

- This is the probability of incorrectly rejecting the null hypothesis.

Type II Error (β)

- The **blue shaded region** to the right of the cutoff is

$$\beta = P(\text{fail to reject } H_0 \mid H_0 \text{ is false}).$$

- This is the probability of failing to detect the true mean $\mu = 297$.

Power ($1 - \beta$)

- Power increases when:
 - the true mean moves farther away from 300,
 - the sample size increases,
 - or the variability (σ) decreases.

This visualization highlights that **power is not a single number**, but depends on *which alternative value* of μ we consider when H_0 is false.

12 Two Sample t -test for Independent and Paired Data

Learning Objectives

1. Distinguish between **independent** and **paired** two-sample t -tests.
2. Identify the **appropriate test** based on study design.
3. State the **model assumptions** for each test.
4. Perform two-sample t -tests in **SAS** using PROC TTEST.
5. Interpret **test statistics, p-values, and confidence intervals**.

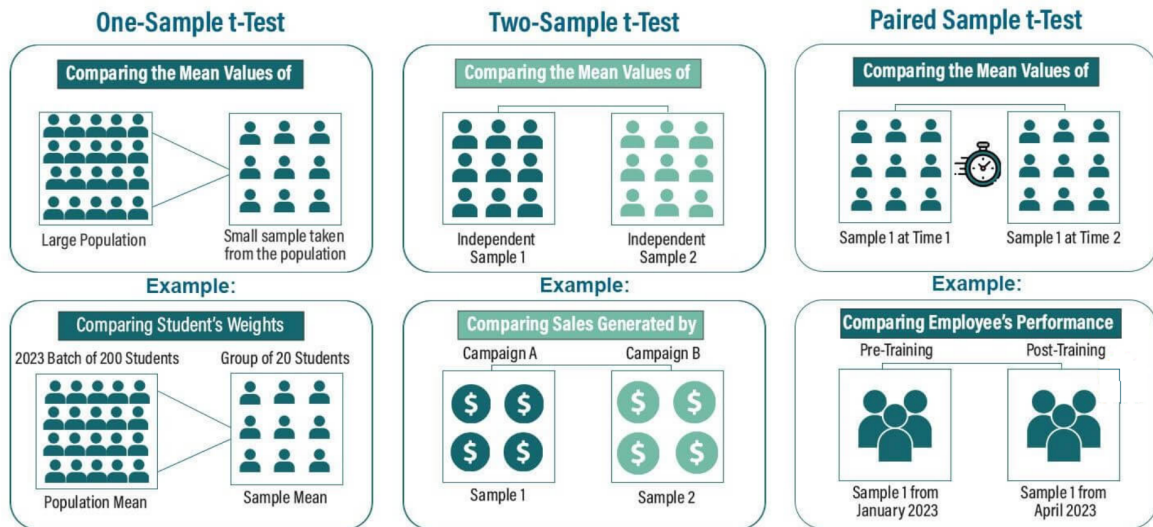
In many applications, we want to compare **two populations**, for example:

- Treatment vs control groups
- Male vs female outcomes
- Before vs after an intervention
- Method A vs Method B

The key question is:

Are the population means different?

This leads to **two-sample inference**.



Since we see that there are different scenarios for the t -test, we may have another question:

Which t -test should I use?

The answer depends on the **study design**. We will discuss two main types of two-sample t -tests:

1. **Independent two-sample t -test:** Used when the two samples are independent of each other (e.g., treatment vs control groups).

```

/*=====
Create the dataset for two-sample t-test example
Variables:
  Species : group indicator (1 or 2)
  Height  : numeric response
=====*/

DATA my_data;
  INPUT Species Height;
  DATALINES;
1 13
1 15
1 15
1 16
1 16
1 16
1 16
1 17

```

```
1 18
1 18
1 19
1 20
1 21
2 15
2 15
2 16
2 18
2 19
2 19
2 19
2 20
2 21
2 23
2 23
2 24
;
RUN;

/* Optional: check the data */
PROC PRINT DATA=my_data;
RUN;
```

Obs	Species	Height
1	1	13
2	1	15
3	1	15
4	1	16
5	1	16
6	1	16
7	1	17
8	1	18
9	1	18
10	1	19
11	1	20
12	1	21
13	2	15
14	2	15
15	2	16
16	2	18
17	2	19
18	2	19
19	2	19
20	2	20
21	2	21
22	2	23
23	2	23
24	2	24

2. **Paired two-sample t -test:** Used when the two samples are related or paired (e.g., before vs after measurements on the same subjects).

Student	Pre-Test Score	Post-Test Score
1	88	91
2	82	84
3	84	88
4	93	90
5	75	79
6	78	80
7	84	88
8	87	90
9	95	90
10	91	96
11	83	88
12	89	89
13	77	81
14	68	74
15	91	92

12.1 Inference Goals of the Two Sample t -Test

Suppose the two population means are denoted by:

$$\mu_1 \quad \text{and} \quad \mu_2.$$

A two-sample t -test always begins with the same **null hypothesis**:

$$H_0 : \mu_1 = \mu_2.$$

That is, the two population means are equal. The alternative hypothesis can take different forms.

Alternative Hypotheses

Depending on the scientific question, the alternative hypothesis can take one of three forms.

1. Two-sided (two-tailed): $H_1 : \mu_1 \neq \mu_2$

The two population means are different.

2. Left-tailed: $H_1 : \mu_1 < \mu_2$

Population 1 has a *smaller* mean than population 2.

3. Right-tailed: $H_1 : \mu_1 > \mu_2$

Population 1 has a *larger* mean than population 2.

Confidence Interval Interpretation

When constructing a confidence interval, inference is made on the **difference of means**: $\mu_1 - \mu_2$.

- If the confidence interval **contains 0**, we fail to reject H_0 .
- If the confidence interval **does not contain 0**, we reject H_0 .

12.2 Sample SAS code

12.2.1 Two Independent Sample *t*-test

```
/*=====
Example: Two-Sample t-Test (Independent Samples)
STAT 8678 - Two-Sample t-Test
=====*/

/*-----
Create the dataset
-----*/
DATA my_data;
  INPUT Species $ Height;
  DATALINES;
1 13
1 15
1 15
1 16
1 16
1 16
1 17
1 18
1 18
1 19
1 20
```

```

1 21
2 15
2 15
2 16
2 18
2 19
2 19
2 19
2 20
2 21
2 23
2 23
2 24
;
RUN;

/*-----
Two-sample t-test
H0: 1 - 2 = 0
Two-sided test,  = 0.05
-----*/
PROC TTEST DATA=my_data
    SIDES=2
    ALPHA=0.05
    H0=0;
    CLASS Species;
    VAR Height;
RUN;

```

The TTEST Procedure

Variable: Height

Species	Method	N	Mean	Std Dev	Std Err	Minimum	Maximum
1		12	17.0000	2.2962	0.6629	13.0000	21.0000
2		12	19.3333	3.0551	0.8819	15.0000	24.0000
Diff (1-2)	Pooled		-2.3333	2.7024	1.1033		
Diff (1-2)	Satterthwaite		-2.3333		1.1033		

Species	Method	Mean	95% CL Mean		Std Dev	95% CL Std Dev	
1		17.0000	15.5410	18.4590	2.2962	1.6266	3.8987
2		19.3333	17.3922	21.2744	3.0551	2.1642	5.1871
Diff (1-2)	Pooled	-2.3333	-4.6213	-0.0453	2.7024	2.0900	3.8249
Diff (1-2)	Satterthwaite	-2.3333	-4.6316	-0.0350			

Method	Variances	DF	t Value	Pr > t
Pooled	Equal	22	-2.11	0.0460
Satterthwaite	Unequal	20.422	-2.11	0.0469

Equality of Variances				
Method	Num DF	Den DF	F Value	Pr > F
Folded F	11	11	1.77	0.3577

Note on Two-Sample t-Test Methods in SAS

In PROC TTEST, SAS reports results from **two different methods** by default:

- **Method: Pooled**

This corresponds to the **two-sample equal-variance t-test**, which assumes $\sigma_1^2 = \sigma_2^2$. The test statistic uses a **pooled variance estimator**.

- **Method: Satterthwaite**

This corresponds to the **two-sample unequal-variance t-test**, also known as the **Welch t-test**.

No assumption of equal variances is required, and the degrees of freedom are approximated using the **Satterthwaite formula**.

Practical guidance:

When the equal-variance assumption is questionable, the **Satterthwaite (Welch) test** is generally preferred and is the default recommendation in modern practice.

12.2.2 Paired Sample *t*-test

```
/* Create dataset */
DATA test_scores;
    INPUT PRE POST;
    DATALINES;
88 91
82 84
84 88
93 90
75 79
78 80
84 88
87 90
95 90
91 96
83 88
89 89
77 81
68 74
91 92
;
RUN;

/*view dataset*/
PROC PRINT DATA=test_scores;

/*perform paired samples t-test*/
PROC TTEST DATA=test_scores ALPHA=.05;
```

```

PAIRED PRE*POST;
RUN;

```

The TTEST Procedure

Difference: PRE - POST

N	Mean	Std Dev	Std Err	Minimum	Maximum
15	-2.3333	3.0394	0.7848	-6.0000	5.0000

Mean	95% CL Mean		Std Dev	95% CL Std Dev	
-2.3333	-4.0165	-0.6502	3.0394	2.2252	4.7935

DF	t Value	Pr > t
14	-2.97	0.0101

12.3 Statistical Assumptions of the Two Sample *t*-Test

The validity of a two-sample *t*-test relies on the following assumptions:

1. **Continuous data**

The response variable is continuous (not discrete or categorical).

2. **Normality**

The data in each population follow a normal distribution.

- In practice, the two-sample *t*-test is reasonably robust to mild departures from normality, especially for moderate or large sample sizes.

3. Equal variances (for the pooled t-test)

The population variances of the two groups are equal:

$$\sigma_1^2 = \sigma_2^2.$$

- If this assumption is violated, the **Welch (Aspin–Welch / Satterthwaite) two-sample t-test** should be used instead.

4. Independence of samples

The two samples are independent of each other.

- There is no relationship between individuals in Sample 1 and Sample 2.
- If observations are paired or matched, a **paired t-test** should be used instead.

5. Random sampling

Both samples are simple random samples from their respective populations.

Each individual in the population has an equal probability of being selected.

12.4 Other Two-Sample Tests

As discussed in the one-population setting, there are several hypothesis tests related to the two-sample problem beyond the two-sample *t*-test. The most common ones include:

12.4.1 Two-Proportion Z Test

This test is used to compare **two population proportions**.

Hypothesis Testing

Let p_1 and p_2 denote the two population proportions.

- **Null hypothesis**

$$H_0 : p_1 = p_2$$

(the two population proportions are equal)

- **Alternative hypotheses**

– Two-tailed:

$$H_1 : p_1 \neq p_2$$

(the two population proportions are not equal)

– Left-tailed:

$$H_1 : p_1 < p_2$$

(population 1 proportion is less than population 2 proportion)

- Right-tailed:

$$H_1 : p_1 > p_2$$

(population 1 proportion is greater than population 2 proportion)

- **Confidence Interval** Inference is made on the difference in proportions:

$$p_1 - p_2.$$

12.4.2 Two-Sample Variance *F* Test

This test is used to compare **two population variances**.

Hypothesis Testing

Let σ_1^2 and σ_2^2 denote the two population variances.

- **Null hypothesis**

$$H_0 : \sigma_1^2 = \sigma_2^2$$

(the two population variances are equal)

- **Alternative hypotheses**

- Two-tailed:

$$H_1 : \sigma_1^2 \neq \sigma_2^2$$

(the two population variances are not equal)

- Left-tailed:

$$H_1 : \sigma_1^2 < \sigma_2^2$$

(population 1 variance is less than population 2 variance)

- Right-tailed:

$$H_1 : \sigma_1^2 > \sigma_2^2$$

(population 1 variance is greater than population 2 variance)

- **Confidence Interval** Inference is made on the ratio of variances:

$$\frac{\sigma_1^2}{\sigma_2^2}.$$

12.4.3 Summary

Test	Parameter of Interest	CI Targets
Two-sample t -test	$\mu_1 - \mu_2$	Difference in means
Two-proportion Z test	$p_1 - p_2$	Difference in proportions
Two-sample F test	σ_1^2 / σ_2^2	Ratio of variances

Part III

Regression Analysis

13 Aanalysis of Variance

Learning Objectives

1. Understand what **ANOVA (Analysis of Variance)** is.
2. Explain **why and when** ANOVA is needed.
3. Recognize the connection between **two-sample t-tests** and **one-way ANOVA**.
4. Formulate ANOVA models and hypotheses.
5. Perform one-way ANOVA in **SAS** and interpret the output.

13.1 Introduction

In the previous lecture, we studied the **two-sample t-test**, where the goal was to compare the means of **two independent populations**.

Recall the equal-variance two-sample model:

$$X_{1i} \stackrel{iid}{\sim} N(\mu_1, \sigma^2), \quad i = 1, \dots, n_1,$$

$$X_{2j} \stackrel{iid}{\sim} N(\mu_2, \sigma^2), \quad j = 1, \dots, n_2,$$

with inference focused on the mean difference

$$\mu_1 - \mu_2.$$

The corresponding hypothesis test was:

$$H_0 : \mu_1 = \mu_2.$$

13.2 From Two-Sample t-Test to ANOVA

A natural question arises:

What if we want to compare more than two population means?

For example: - Comparing test scores across **three or more teaching methods** - Comparing average yields across **multiple fertilizer types** - Comparing mean response times across **several algorithms**

Running multiple pairwise t-tests is **not appropriate**, because: - It inflates the **Type I error rate** - It leads to inconsistent inference

This motivates the need for **ANOVA**.

13.3 ANOVA as a Generalization of the Two-Sample t-Test

To unify the two-sample setting, we can rewrite the data using an **indicator variable**.

Let the combined data be:

$$\{(I_{i,2}, Y_i)\}_{i=1}^{n_1+n_2},$$

where

$$I_{i,2} = \begin{cases} 1, & \text{if observation } i \text{ comes from group 2,} \\ 0, & \text{otherwise.} \end{cases}$$

Then the two-sample model can be written as a **linear model**:

$$Y_i = \alpha + \beta_1 I_{i,2} + \varepsilon_i,$$

where

$$\varepsilon_i \stackrel{iid}{\sim} N(0, \sigma^2).$$

- α represents the mean of group 1
- $\beta_1 = \mu_2 - \mu_1$
- Testing $\mu_1 = \mu_2$ is equivalent to testing:

$$H_0 : \beta_1 = 0$$

13.4 Extending to Multiple Groups

Suppose now we have $K > 2$ groups.

ANOVA extends this idea by modeling:

$$Y_{ij} = \mu + \tau_j + \varepsilon_{ij},$$

where: - μ is the overall mean - τ_j is the effect of group (j) - $\varepsilon_{ij} \stackrel{iid}{\sim} N(0, \sigma^2)$

The **ANOVA null hypothesis** is:

$$H_0 : \mu_1 = \mu_2 = \cdots = \mu_K,$$

or equivalently,

$$H_0 : \tau_1 = \tau_2 = \cdots = \tau_K = 0.$$

13.5 Why ANOVA Works

ANOVA answers a single question:

Is the variability *between* group means larger than the variability *within* groups?

This is formalized using the **F-statistic**, which compares:

- **Between-group variation**
- **Within-group variation**

If between-group variation is sufficiently large relative to within-group variation, we reject H_0 .

13.6 What's Next

In the remainder of this lecture, we will cover:

- The ANOVA decomposition:
 - Total Sum of Squares
 - Between-Group Sum of Squares
 - Within-Group Sum of Squares
- The ANOVA table
- One-way ANOVA in **SAS** using `PROC ANOVA` and `PROC GLM`
- Post-hoc comparisons (Tukey, Bonferroni)

Key takeaway:

The two-sample t-test is a *special case* of ANOVA.

ANOVA provides a unified framework for comparing **multiple means simultaneously** while controlling error rates.

Part IV

Hands on Class

14 Writing Macro in SAS

Learning Objectives

In this hands-on session, we will practice writing **SAS macros** through a concrete example.

The goal is to help you develop a **systematic workflow** for turning regular SAS code into reusable and flexible macro functions.

By the end of this activity, you should be able to:

1. Understand the logic behind SAS macro programming.
2. Identify macro inputs and outputs.
3. Write simple SAS macros for statistical analysis.
4. Apply macros to automate repetitive tasks.

14.1 Guideline: How to Write SAS Macros

Based on experience, a reliable way to write SAS macros is to follow **three steps**:

Step 1: Write Regular SAS Code First

- Write SAS code that produces the desired result **without using macros**.
- Clearly identify:
 - What the code does
 - Which values may change (dataset name, variable name, parameters, etc.)

Step 2: Introduce Macro Variables Using %LET

- Replace hard-coded values with **macro variables** using %LET.
- This makes the code flexible and easier to generalize.

Step 3: Wrap the Code into a Macro

- Convert the %LET variables into **macro arguments**.
- Place the code inside %MACRO ... %MEND.
- Remove the %LET statements.
- Your macro now behaves like a function.

This three-step strategy is also useful when writing functions in **R** or **Python**.

14.2 Practice Examples in this activity

To practice these steps, in this activity, we will:

1. Write the first macro to compute the **sample mean**, **standard deviation** as well as the p -value
2. Write the second macro to conduct a **one-sample Z-test**

Eventually, you will practice the three-step workflow **twice** through those two macros

14.2.1 Dataset for this exercise: Court Length Data

We reuse the court length example from previous lecture.

```
DATA time;
  INPUT time @@;
  DATALINES;
    43 90 84 87 116 95 86 99 93 92
    121 71 66 98 79 102 60 112 105 98
  ;
RUN;
```

14.3 First SAS Macro for Reporting Mean and Standard Deviation

Step 1: Write Regular SAS Code (No Macros)

First, write SAS code that computes the **sample mean**, **sample size**, and **standard deviation** from a dataset.

Key: Replace ? and ?? with the dataset name and variable name.

```
PROC MEANS DATA=?;
  VAR ??;
  OUTPUT OUT=zdata
    MEAN=Mean
    N=n
    STDDEV=Std_Dev;
RUN;
```

At this stage:

- Do not worry about macros.
- Focus only on producing the correct output.

Step 2: Introduce Flexibility Using %LET

Next, make the code flexible by replacing hard-coded values with macro variables.

```
%LET D = time;    /* dataset name */
%LET V = time;    /* variable name */

PROC MEANS DATA=&D;
  VAR &V;
  OUTPUT OUT=zdata
         MEAN=Mean
         N=n
         STDDEV=Std_Dev;
RUN;
```

Key idea:

- %LET creates macro variables D and V
- &D controls which dataset is used
- &V controls which variable is analyzed

This allows the same code to work for any dataset and variable.

Step 3: Change everything to a MACRO syntax

Finally, wrap the code into a macro function, that is, replace %LET variables with macro arguments and place the code inside %MACRO ... %MEND.

```
%MACRO myFunction(input1, input2);
...
%MEND myFunction;

%myFunction(input1, input2);
```


14.4 Second SAS Macro for creating confidence interval and p-value from a Z-test

In this section, we apply the **three-step macro writing strategy** again to build a SAS macro that computes:

- a $(1 - \alpha)100\%$ **confidence interval** for the population mean, and
- the **Z-test statistic and p-value** for testing

$$H_0 : \mu = \mu_0.$$

You should **think carefully through each step** before combining everything into a macro.

You may need to utilize the following SAS functions:

Code chunk 1: Dataset

```
DATA time;
  INPUT time @@;
  DATALINES;
    43 90 84 87 116 95 86 99 93 92
    121 71 66 98 79 102 60 112 105 98
  ;
RUN;
```

Code chunk 2: Mean, standard deviation, and sample size calculation

```
PROC MEANS DATA=time;
  VAR time;
  OUTPUT OUT=zdata
    MEAN=Mean
    N=n
    STDDEV=Std_Dev;
RUN;
```

For this code chunk, you may verify that:

- Mean, Std_Dev, and n are correctly stored in the dataset zdata.

Code chunk 3: CI and Z-test Quantities

```

DATA ztest;
    SET zdata;

    /* Confidence level */
    CL = (1 - &alpha) * 100;

    /* Confidence interval */
    ZLower = Mean - probit(1 - &alpha/2) * Std_Dev / sqrt(n);
    ZUpper = Mean + probit(1 - &alpha/2) * Std_Dev / sqrt(n);

    /* Z-test statistic */
    zts = (Mean - &muzero) / (Std_Dev / sqrt(n));
RUN;

```

This code chunk creates:

- confidence level,
- confidence interval bounds,
- Z-test statistic.

Notes:

- &alpha is the significance level
- &muzero is the hypothesized mean under H_0
- probit() returns the standard normal quantile

Step 3: Introduce Macro Variables for Flexibility

Now, think about what should be user-controlled inputs:

- dataset name
- variable name
- significance level α
- null mean μ_0

Example macro variables:

```

%let D = time;
%let V = time;
%let alpha = 0.05;
%let muzero = 90;

```

Update your code by replacing fixed values with macro variables.

14.5 Convert Everything into a Macro

Finally, combine everything we learned from this activity into a reusable SAS macro.

Step 1

```
DATA TIME;
    INPUT TIME @@;
    DATALINES;
43 90 84 87 116 95 86 99 93 92
121 71 66 98 79 102 60 112 105 98
;
RUN;

PROC MEANS DATA=TIME;
    VAR TIME;
    OUTPUT OUT=ZDATA MEAN=MEAN N=N STDDEV=STD_DEV;
RUN;

DATA ZTEST;
    SET ZDATA;
    CL = (1 - &ALPHA) * 100;
    ZLOWER = MEAN - PROBIT(1 - &ALPHA / 2) * STD_DEV / SQRT(N);
    ZUPPER = MEAN + PROBIT(1 - &ALPHA / 2) * STD_DEV / SQRT(N);
    ZTS = (MEAN - &MUZERO) / (STD_DEV / SQRT(N));
    IF &SIDE = 'L' THEN ZPVALUE = PROBNORM(ZTS);
    IF &SIDE = 'U' THEN ZPVALUE = 1 - PROBNORM(ZTS);
    IF &SIDE = 'B' THEN ZPVALUE = 2 * MIN(1 - PROBNORM(ZTS), PROBNORM(ZTS));
RUN;

PROC PRINT DATA=ZTEST NOOBS;
    VAR N MEAN STD_DEV ZLOWER ZUPPER ZPVALUE;
    TITLE 'Z-INTERVALS AND Z-TEST';
RUN;
```

Step 2

```
%LET ALPHA = 0.05;    *VALUE OF ALPHA;
%LET MUZERO = 80;
%LET SIDE = 'B';

PROC MEANS DATA=TIME;
```

```

VAR TIME;
OUTPUT OUT=ZDATA MEAN=MEAN N=N STDDEV=STD_DEV;
RUN;

DATA ZTEST;
SET ZDATA;
CL = (1 - &ALPHA) * 100;
ZLOWER = MEAN - PROBIT(1 - &ALPHA / 2) * STD_DEV / SQRT(N);
ZUPPER = MEAN + PROBIT(1 - &ALPHA / 2) * STD_DEV / SQRT(N);
ZTS = (MEAN - &MUZERO) / (STD_DEV / SQRT(N));
IF &SIDE = 'L' THEN ZPVALUE = PROBNORM(ZTS);
IF &SIDE = 'U' THEN ZPVALUE = 1 - PROBNORM(ZTS);
IF &SIDE = 'B' THEN ZPVALUE = 2 * MIN(1 - PROBNORM(ZTS), PROBNORM(ZTS));
RUN;

PROC PRINT DATA=ZTEST NOOBS;
VAR N MEAN STD_DEV ZLOWER ZUPPER ZPVALUE;
TITLE 'Z-INTERVALS';
RUN;

```

Step 3

```

DATA TIME;
INPUT TIME @@;
DATALINES;
43 90 84 87 116 95 86 99 93 92
121 71 66 98 79 102 60 112 105 98
;
RUN;

%MACRO ZTEST_SELF(D, V, ALPHA, MUZERO, SIDE);

PROC MEANS DATA=&D;
VAR &V;
OUTPUT OUT=ZDATA MEAN=MEAN N=N STDDEV=STD_DEV;
RUN;

DATA ZTEST;
SET ZDATA;
CL = (1 - &ALPHA) * 100;
ZLOWER = MEAN - PROBIT(1 - &ALPHA / 2) * STD_DEV / SQRT(N);
ZUPPER = MEAN + PROBIT(1 - &ALPHA / 2) * STD_DEV / SQRT(N);

```

```

ZTS = (MEAN - &MUZERO) / (STD_DEV / SQRT(N));
IF &SIDE = 'L' THEN ZPVALUE = PROBNORM(ZTS);
IF &SIDE = 'U' THEN ZPVALUE = 1 - PROBNORM(ZTS);
IF &SIDE = 'B' THEN ZPVALUE = 2 * MIN(1 - PROBNORM(ZTS), PROBNORM(ZTS));
RUN;

PROC PRINT DATA=ZTEST NOOBS;
  VAR N MEAN STD_DEV CL ZLOWER ZUPPER ZPVALUE;
  TITLE 'Z-INTERVALS & TESTS';
RUN;

%MEND ZTEST_SELF;

%ZTEST_SELF(TIME, TIME, 0.05, 80, 'B');

```

Example Macro Call

```
%ZTEST_SELF(TIME, TIME, 0.05, 80, 'B');
```

- TIME (first): input dataset
- TIME (second): variable of interest
- 0.05: significance level
- 80: hypothesized mean
- 'B': two-sided Z-test
- 'L' = left-tailed
- 'U' = right-tailed
- 'B' = two-tailed

Summary

By following the three-step macro workflow, you have:

1. Written correct and verifiable SAS code,
2. Introduced flexibility using macro parameters,
3. Built a reusable SAS macro for Z-test inference.

This same workflow applies directly to:

- t-tests,
- confidence intervals,
- ANOVA summaries,
- regression diagnostics.

Once you master this pattern, writing SAS macros becomes systematic, reusable, and scalable, just like functions in R or Python.

References